dbdome Specification

DBDOME_dbanalytics_service - Specification - Functionalities - Per-panel SQL

Generated 2026-06-30

DBDOME*dbanalytics*service - Comprehensive Specification

> Derived from the live `dbanalytics` PostgreSQL catalog, the `DBDOME_dbanalytics_service` codebase, the Grafana deployment under `C:\ProgramData\DBDOME\bin`, and the `config` schema. Generated 2026-06-30.

1. Purpose

DBDOME is an agentless **database security, governance/compliance (GRC), performance and health monitoring** platform. It continuously interrogates heterogeneous database servers, maps findings to a large **root-cause catalog** (~7,000 root causes across Security, Performance and Health), **alerts** (email / SIEM), **enforces data-protection** (masking, anonymization, session blocking), runs a full **GRC program** (compliance reporting, attestation, segregation-of-duties, risk register, immutable audit hash chain), and surfaces everything through **91 Grafana dashboards**.

Supported monitored engines: **Oracle, SQL Server, PostgreSQL, MySQL, MariaDB, Informix**.

2. Runtime architecture

Component	Detail
Web service	`DBDOME_web` -> `dbdome_service.exe --service web`; FastAPI + uvicorn; **198 HTTP routes**; ports **8080** and **8001** (`config.global_params.dbexpert.ai.api.port`)
Scheduler service	`DBDOME_scheduler` -> `dbdome_service.exe --service scheduler`; APScheduler; job set driven by `metrics.registered_processes` (live toggle via a 30 s config-watcher)
Grafana	`DBDOME_Grafana` (NSSM) -> `dbdomeg-server.exe`, port **8000** (`config.global_params.grafana.dbexpert.ai.port`); SQLite `grafana.db` under `...\DBDOME\data\`
Catalog DB	PostgreSQL database **`dbanalytics`** (owner `dbdome_adm`; read role `dbdome_mon_usr`), **21 schemas**
Entry points	`dbdome_main.py` (run/orchestrator), `dbdome_service.py` (Windows-service wrapper), `http_server.py` (FastAPI app), `job_operation_scheduler.py` (dynamic job resolver)
Build/deploy	PyInstaller onedir (`DBDOME_dbanalytics.spec`, built with `C:\apps\venv314`) -> `C:\ProgramData\DBDOME\bin`; packaged for `C:\installs\dbdome_setup` (full install) and `C:\installs\dbdome_update` (incremental updater). Ubuntu appliance runs the same code from source.
Cloud ingest	Optional push to `config.global_params.INGEST_URL` (`https://dbexpertai.com/api/ingest`) with `INGEST_API_KEY`

3. Data model - `dbanalytics` schemas

Schema	Tables	Views	Purpose

`rootcause`	20	4	Detection catalog: domains, areas, issues, root*causes, detection*paths/steps, resolution*paths/steps, vendors, risk*level. View `v_rootcauses` flattens runnable detections.
`metrics`	27	5	Monitored `servers`, `custom_metrics`, `registered_processes` (scheduler jobs), server logs. View `v_custom_metrics` = custom_metrics ∪ rootcause detection SQL.
`monitoring`	72	858	Per-root-cause result views over `general_metric_metadata_results` (partitioned jsonb store); helper fns (`get_local_ip`, `jsonb_lower_keys`, `get_alert_log_resultset_byid`).
`config`	36	6	**All feature configuration** (see §10).
`alerts`	4	-	`alert_log`, `mail_alert_log`, blocks.
`anonymization`	28	-	Anonymized table copies + masking/tokenization state.
`log`	14	25	Operation log, DDL audit, firewall audit, SoD violations, RBAC/masking/DP logs.
`threats`	8	-	Behavioural analytics / risk-scoring state.
`processes`	5	5	Detection-tree process runs and history.
`jobs`	5	-	Job/report scheduling state.
`reports`	2	-	Rendered HTML/PDF report store.
`widget`	20	-	Dashboard widget definitions/exports.
`siem` / `siem_config`	1 / 1	9	SIEM forwarding state and views.
`users` / `web` / `meta` / `public` / `action` / `flowchart` / `monitoring_history` / `reports`	...	...	Auth, web state, migration ledger (`meta.schema_migrations`), shared objects.

4. Collection & detection engine

- **Vendor collectors** (`collection/<vendor>/monitoring_metrics_<vendor>_generic_query.py`) for **oracle, mssql, postgres, mysql, MariaDB, informix** (+ `purge`). Each: decrypt per-server secret -> connect with the production connector -> run each metric query -> `read_sql` -> serialize via `utils.metric_json.to_records_json` (datetimes rendered as `YYYY-MM-DD HH:MM:SS`, not epoch) -> `_build_comparison` vs `expected` -> store + conditionally alert.
- **Root-cause catalog** (live counts): **Health 2,714 · Performance 2,372 · Security 1,950** root causes, under 3 enabled domains (`HLTH`, `PERF`, `SEC`). Each root cause has per-vendor **detection paths** with one or more **steps** (multi-step `live -> audit` trees, e.g. `SEC-SQL-AUD-020`), and **resolution paths/steps** with a `risk_level`.
- **Detection-tree executor** (`processes/detection_tree_executor.py`) walks multi-step paths (`on_match_action` / `on_no_match_action` = `confirmed` | `ruled_out` | `next`) for diagnosis/evidence; the generic collector runs the flat metric.
- Collection cadence is split by domain: `collect_metrics_operation_sec` (30 s), `_perf` (10 s), `_hlth` (10 s), `_critical` (30 s), `_other` (60 s).

---

5. Alerting & routing - `config.webook_alerts`

Routing is a matrix keyed by **(metric*type* = *domain, risk*level)** with per-cell booleans:

Domain	Risk	mail	SIEM	diagnosis evidence	blocker	auto_mask
Security	critical	[x]	[x]	[x]	-	[x]
Security	high	[x]	-	-	-	-
Performance	critical/high	-	[x]	[x]	-	-
Health	high/med/low	-	[x]	[x]	-	-

Plus `is_active` and `recurrency_hours` (mail de-dup window). Mail is gated additionally by `rootcause.risk_level.is_active`. Delivery:

- **Email** - `email_utils.smtp_email_sender.send_mail_alert_no_attachment` reads `config.mail_config` ■ `config.mail_groups`; logs to `alerts.mail_alert_log` (72 h dedup, SMTP circuit-breaker). The alert body renders `metric_metadata_json` as an HTML table.
- **SIEM** - `config.siem` (e.g. `rapid_7`), Rapid7 + CrowdStrike senders.
- **Diagnosis evidence** - `manage_diagnosys_alerts` attaches the reproduction query/results.
- Findings written to `alerts.alert_log` (one per server+root_cause per hour), surfaced by `monitoring.get_alert_log_resultset_byid` for the Grafana alert-detail panel.

---

6. Data protection & active response

Capability	Engine	Config / gate
Dynamic Data Masking	`processes/data_masking_engine.py`, `auto_mask.py`	`config.masking_rules`; `webook_alerts.auto_mask`; `global_params.masking_dry_run`
Static masking / anonymization / tokenization	`processes/data_protection.py`	`anonymization` schema; `config.masking_tokens`
Session **blocker** (kill offending sessions)	`processes/blocker.py`	`webook_alerts.blocker`; `global_params.blocker_dry_run`
Encryption / tokenize columns	`data_protection.py` (`encrypt_column`)	`global_params.encryption_dry_run`
RBAC provisioning	`processes/rbac_provisioning.py`	`/rbac_provision`; `log.rbac_log`

Threat response (escalate/notify/suspend/block-IP)	`processes/threat_response_engine.py`	`config.threat_response_playbooks`, `config.blocked_ips`, `config.suspended_users`

All destructive actions honour **dry-run flags** in `config.global_params`.

---

7. GRC / compliance program

Regulations in scope (`config.regulation_profiles`): **GDPR, HIPAA, PCI-DSS, SOC2** (+ PPL dashboard). Engines (each a scheduler process):

- **Firewall policy engine** - `config.firewall_policies` (56), `grc_firewall_scanner.py` -> `log.firewall_audit_log`.
- **Immutable audit hash chain** - `hash_chain_writer.py` (60 s) + `hash_chain_verifier.py` (daily) SHA-256 chain over audit rows.
- **Segregation of Duties** - `sod_scanner.py` vs `config.sod_rules` -> `log.sod_violations`.
- **Risk register** - `config.risk_register` (inherent/residual scoring).
- **Control attestation** - `attestation_scheduler.py`, `config.attestation_controls` / `attestation_periods`.
- **Compliance reporting** - `compliance_report_generator.py`: daily PCI-DSS/HIPAA, weekly GDPR/SOC2 PDFs (`config.compliance_report_schedules`).
- **Cross-border transfer tracking** - `cross_border_scanner.py`, `config.data_regions` / `transfer_agreements`.
- **Access review** - `access_review_scheduler.py`, `config.access_review_cycles`.
- **Incident lifecycle / GDPR 72 h** - `incident_gdpr_timer.py`.
- **User risk scoring** - `user_risk_scoring.py` (behaviour baseline, `threats` schema).
- **DDL audit** - `ddl_audit_scanner.py` -> `log.ddl_audit_log`.
- **Policy-exception notifier**, **evidence-package generator**, **continuous data discovery / sensitive-schema discovery** (`config.masking_rules` sync), **TLS enforcement** & **vulnerability scanners**.

---

8. Scheduler processes (`metrics.registered_processes`)

Process	Interval	Active (this catalog)	Role
collect*metrics*operation_sec / _perf / _other	30 / 10 / 60 s	[x]	Domain collection
collect*metrics*operation_hlth / _critical	10 / 30 s	-	Domain collection
gmmr_maintain	3600 s	[x]	Partition maintenance + dedup of results store
auto_mask	300 s	[x]	Apply DDM when a critical alert has `auto_mask`
blocker	120 s	[x]	Kill sessions for `blocker` root causes (dry-run gated)
dump*and*prune_metrics	86400 s	[x]	CSV dump + retention prune (`config.dump_metrics`)
retention*space*alert	3600 s	[x]	Email when free disk < `retention_free_space_alert_pct`

grc*firewall*scan / hash*chain*write / hash*chain*verify	60 / 60 / 86400 s	-	GRC phase 1
user*risk*scoring / compliance*reports** / sync*masking*rules / run*threat*response / sod*violation*scan / cross*border*scan / attestation*scheduler* / *incident*gdpr*timer* / *ddl*audit_scan	various	-	GRC phases 2-6
alerts, analyse*metrics*operation, detection*tree*build/oracle, dashboard*data*export, report*job,* *metrics*advisories, ...	-	-	Optional auxiliary jobs

(Toggle any by `UPDATE metrics.registered_processes SET is_active=...;` effective within 30 s.)

---

9. Web / API (`http_server.py`, 198 routes)

Functional groups: configuration forms (`/processform`, `/serverform`, `/mailconfiguration`, `/siem_configuration`, `/global_param_set`, `/webook_alert_set`, `/custom_metrics`, `/addrecipients`), reporting (`/generate_report`, `/capture_report`, `/reportbymail`, `/download_report`), data-protection actions (`/dp_dynamic_mask`, `/dp_static_mask`, `/dp_anonymize[_pre|_actual|_confirm|_preview]`, `/rbac_provision`, `/sensitive_column_add`), server lifecycle (`/server_enable`), and the rootcause/GRC admin APIs + the cloud ingest endpoints.

---

10. Configuration reference - the `config` schema (feature switches)

Table	Drives
`global_params`	Global keys: ports (api 8001, grafana 8000), `local_ip`, dry-run flags (`blocker_/masking_/encryption_dry_run`), `dump_location`, `retention_free_space_alert_pct`, `firewall_audit_retention_days`, `INGEST_URL`/`INGEST_API_KEY`, dashboard/data paths
`webook_alerts` (12)	Alert routing matrix (mail/SIEM/diagnosis/blocker/auto_mask per domainxrisk)
`mail_config` / `mail_groups` / `mail_alerts` / `mail_alerts_schedule` / `mail_jobs`	SMTP server + recipient groups + scheduled mailings
`siem` / `sime_interface`	SIEM target(s) (Rapid7, CrowdStrike) and interface keys
`thresholds` / `alerts` / `alerts_thresholds` / `alerts_reports` / `alerts_issue_root_causes`	Threshold definitions and alert↔report↔rootcause links
`reports` (151) / `reports_jobs`	Report catalog (query + URL) and scheduling
`retention_policy` (5222) / `dump_metrics` / `dumps`	Per-server/table retention + dump locations
`firewall_policies` (56) / `sqli_signatures` (13)	Firewall rules + SQL-injection signatures
`masking_rules` / `masking_tokens`	DDM rules + tokenization vault
`regulation_profiles` (4) / `compliance_report_schedules` (16)	GDPR/HIPAA/PCI-DSS/SOC2 + report schedules

`sod_rules` (4) / `risk_register` / `attestation_controls` (13) / `attestation_periods`	SoD, risk, attestation
`data_regions` / `transfer_agreements` / `access_review_cycles`	Cross-border + access review
`threat_response_playbooks` (7) / `blocked_ips` / `suspended_users`	Automated response
`action_types`	Catalog of remediation actions

Views: `v_blocked_ips_active`, `v_suspended_users_active`, `v_masking_coverage`, `v_masking_rules_detail`, `v_unmasked_sensitive_columns`, `v_mail_alert_schedule`.

---

11. Grafana (91 dashboards)

Grouped by capability:

- **Discovery & PII** - Data Discovery & Classification; PII Sensitive columns / data in use; Credentials stored in tables; Linked-servers/DBLINK hidden credentials.
- **Activity & Audit** - Activity Monitoring & Audit; Schema DML Tracking; Audit expired; Connections per user; Database restored.
- **Threat & Injection** - Threat Detection & Behavioral Analytics; SQL Injection (boolean/comment based); IPS Dashboard (FortiAnalyzer); Unknown TCP connections.
- **Policy & Protection** - Policy Enforcement & Protection; Policy not enforced; Data Protection; Enabled sysadmin; Locked accounts.
- **Vulnerability & Automation** - Vulnerability Assessment; Automation & Workflows; Blocker Activity; Blocking transactions.
- **GRC suite (15)** - Overview, Compliance, Audit Log, Control Attestation, Cross-Border Transfer, Immutable Audit Hash Chain, Incident Lifecycle, Masking Coverage, Policy Exceptions, Risk Register, Risk Scoring, Threat Response, Segregation of Duties, Access Review.
- **Ops & Config** - Rootcauses, Recent alerts, Alerts, Retention (+ policy), Processes, Custom metrics, Configuration, Email/Mail, SIEM configuration, Connectivity, PPL (Israeli Protection-of-Privacy Law).

Datasource: the PostgreSQL `dbanalytics` catalog; dashboard host links are templated via the `${global_ip}` variable resolved from `config.get_local_ip()` (kept in sync with the machine LAN IP).

> **See also:** for the exhaustive per-dashboard breakdown - every dashboard's **panels (title + type), action buttons, links and drill-downs** - see **§17 "Grafana dashboards - full panel / button / link catalog"** in `DBDOME_FUNCTIONALITIES.md`, and **Appendix A "Per-panel SQL/query reference"** for the SQL behind each panel.

---

12. Integrations

- **SIEM**: Rapid7 InsightIDR, CrowdStrike (config.siem).
- **SMTP**: any server via config.mail_config (TLS).
- **SSRS**: report download/attach (`ssrs` package).
- **Cloud ingest**: `dbexpertai.com/api/ingest` with API key.
- **FortiAnalyzer / IPS**: dashboard integration.

---

13. Feature list (summary)

Monitoring & detection

- Agentless multi-vendor monitoring (Oracle, SQL Server, PostgreSQL, MySQL, MariaDB, Informix)

- ~7,000 root-cause detections across Security / Performance / Health
- Multi-step detection trees (live activity + audit-trail), per-vendor SQL
- Custom metrics; per-domain collection cadence; partitioned result store with auto-maintenance
- Human-readable timestamps in stored results

Alerting

- Routing matrix per domainxrisk (email, SIEM, diagnosis evidence, blocker, auto-mask)
- Email (SMTP, recipient groups, dedup, circuit-breaker, HTML result table)
- SIEM forwarding (Rapid7, CrowdStrike); diagnosis-evidence attachment
- Threshold and recurrency controls

Data protection & response

- Dynamic + static masking, anonymization, tokenization, column encryption
- Auto-mask on critical findings; session blocker (kill); RBAC provisioning
- Threat-response playbooks: escalate / notify / suspend user / block IP
- Global dry-run safety switches

GRC / compliance

- GDPR, HIPAA, PCI-DSS, SOC2 (+ PPL) profiles
- Firewall policy engine; SQL-injection signatures
- Immutable SHA-256 audit hash chain (+ verification)
- Segregation-of-Duties, risk register, control attestation
- Compliance PDF reporting (scheduled); cross-border transfer tracking; access reviews
- Incident lifecycle / GDPR 72-h breach timer; DDL audit; user risk scoring

Platform

- FastAPI web (198 routes) + APScheduler (live-toggle jobs)
- 91 Grafana dashboards
- Retention/dump engine with disk-space alerting
- Cloud ingest; PyInstaller packaging (setup + incremental updater); Ubuntu appliance
- IP self-service (`dbdome-setip`) and ORG*IP/local*ip auto-sync on the appliance

DBDOME - Complete Functionalities Catalog

> Exhaustive list of every functionality, enumerated from the 198 HTTP routes (`http_server.py`), the 35 scheduler/process engines (`processes/`), the vendor collectors, and the `config`-schema capabilities. Generated 2026-06-30.

Legend: **[UI]** web page · **[API]** JSON endpoint · **[JOB]** scheduler process · **[ENG]** background engine · **[CLI]** appliance tool.

1. Platform & home

- **[UI]** Landing / home console - `/`, `/dbdome`
- **[UI/API]** Fleet overview (all monitored servers, health) - `/fleet`, `/api/fleet`
- **[API]** Test a database connection (pre-flight) - `/test-connection`
- **[CLI]** Appliance network/IP self-service - `dbdome-setip` (+ console auto-launch)
- **[ENG]** ORG_IP / `config.global_params.local_ip` / Grafana `${global_ip}` auto-sync to LAN IP

2. Monitored-server management

- **[UI]** Add/edit a monitored server - `/serverform`, `/submit-serverform`
- **[UI]** Enable/disable a server - `/server_enable`
- **[UI]** Database restore action - `/db_restore`
- Per-server encrypted credentials in `metrics.servers`; vendors: Oracle, SQL Server, PostgreSQL, MySQL, MariaDB, Informix

3. Metrics & custom metrics

- **[UI]** Define/edit custom metrics - `/custom_metrics`, `/custom_metrics_update`, `/submit_custom_metrics`, `/submit_custom_metrics_update`
- **[UI]** Enable a custom metric / per-server / all-servers - `/submit_custom_metric_enable`, `/metric_enable`, `/metric_enable_all_servers`, `/submit_metrics_enable`, `/submit_metrics_enable_all_servers`
- **[UI]** Edit a metric's schema column mapping - `/update_schema_column`
- **[JOB]** Domain collection cycles - `collect_metrics_operation_sec` (30s), `_perf` (10s), `_hlth`, `_critical`, `_other` (60s)
- **[ENG]** Per-vendor collectors run detection SQL -> serialize (timestamps formatted) -> compare vs expected -> store in `monitoring.general_metric_metadata_results`
- **[JOB]** `gmmr_maintain` - partition maintenance + duplicate cleanup of the results store
- **[JOB]** `dedup_metric_results`, `purge_general_metric_metadata`, `analyse_metrics_operation`

4. Root-cause catalog & detection

- ~7,000 root causes across **Security / Performance / Health** (3 enabled domains)
- **[UI]** Activate/deactivate a root cause - `/root_cause_active`
- **[UI]** Set a root cause's risk level - `/root_cause_risk`
- **[UI]** Manage global risk levels & toggles - `/risklevel`, `/risklevel_toggle`, `/submit-risklevel`, `/api/risk-level/list`
- Multi-step detection paths (live activity + audit trail) per vendor
- **[ENG]** Detection-tree executor - `processes/detection_tree_executor.py`; **[JOB]** `detection_tree_build`, `detection_tree_oracle`
- **[ENG]** Advisories - `advisories/` (`metrics_advisory`, `dashboard_advisories`)

5. Scheduler / process control

- **[UI]** View/toggle background processes - `/processform`, `/submit-processform`
- Live job toggling via `metrics.registered_processes.is_active` (effective <=30s)
- **[ENG]** `process_handler.py` orchestrates registered processes

6. Alerting

- **[UI/API]** Alert dashboard - `/alert_dashboard`, `/api/alert_dashboard`
- **[UI/API]** Alert rules CRUD - `/alert_rules`, `/api/alert-rules`, `/submit-alert-rule`, `/api/delete-alert-rule`
- **[UI/API]** Alert thresholds CRUD - `/alert_thresholds`, `/api/alert-thresholds`, `/submit-alert-threshold`, `/api/delete-threshold`
- **[UI/API]** Webhook / routing alerts CRUD (the domainxrisk matrix) - `/webhook_alerts`, `/api/webhook-alerts`, `/webook_alert_set`, `/submit-webhook-alert`, `/api/delete-webhook-alert`
- **[ENG]** Alert dispatch: email + SIEM + diagnosis-evidence per `config.webook_alerts`
- Findings -> `alerts.alert_log`; mail -> `alerts.mail_alert_log` (72h dedup, SMTP circuit-breaker, HTML result-table body)
- **[UI/API]** GRC alert channels CRUD + delivery log - `/grc/alert-channels`, `/api/grc-alerts/channels` (GET/POST/PUT/DELETE), `/api/grc-alerts/delivery-log`

7. Email / SMTP

- **[UI]** Mail server configuration - `/mailconfiguration`, `/email_configuration`, `/submit-mailconfiguration`, `/submit_email_configuration`
- **[UI]** Recipient groups - `/addrecipients`, `/submit-addrecipients`
- **[API]** Mail config & group management - `/api/mail-config/list`, `/api/mail-config/delete`, `/api/mail-group/delete`
- **[UI/API]** Email panel (send a dashboard panel) - `/api/email-panel`, `/api/email-panel-form`, `/api/print-panel`

8. Reports

- **[UI]** Report catalog & editor - `/reports`, `/reports/edit`, `/reports/save`, `/api/reports/list`, `/api/toggle-report`
- **[UI]** Report scheduling - `/report_schedule`, `/submit-report-schedule`, `/api/report-schedules`
- **[UI]** Generate / capture / download / email a report - `/generate_report`, `/submit-generate_report`, `/capture_report`, `/submit-report_capture`, `/download_report`, `/reportbymail`, `/submit-reportbymail`
- **[ENG]** PDF/HTML rendering - `ReportGenerator/`, dashboard capture (`print_utils/`)

9. Data protection & active response

- **[UI]** Dynamic masking - `/dp_dynamic_mask`, `/mask_column`; revert - `/dp_revert_mask`
- **[UI]** Static masking - `/dp_static_mask`
- **[UI]** Anonymization (3-phase) - `/dp_anonymize`, `/dp_anonymize_pre`, `/dp_anonymize_actual`, `/dp_anonymize_confirm`, `/dp_anonymize_preview`
- **[UI]** Tokenization (phased) - `/dp_tokenize`, `/dp_tokenize_pre`, `/dp_tokenize_actual`, `/dp_revert_tokenize`
- **[UI]** Column encryption - `/encrypt_column`, `/revert_encryption`
- **[UI]** RBAC provisioning - `/rbac_provision`
- **[UI]** Generic remediation action - `/take_action`
- **[API]** Masking rules CRUD + sync - `/api/masking-rules` (GET/POST/PUT), `/api/masking-rules/sync`, `/grc/masking-rules`
- **[JOB]** `auto_mask` - apply DDM when a critical alert has `auto_mask=true`
- **[JOB/ENG]** `blocker` - kill offending sessions (gated by `blocker_dry_run`)
- **[ENG]** `data_masking_engine`, `data_protection`, `rbac_provisioning` (dry-run flags for mask/encrypt/block)

10. Sensitive-data discovery / PII

- **[UI]** Sensitive schema & columns - `/sensitive_schema`, `/sensitive_column_add`
- **[API]** Discover / toggle sensitive columns - `/api/sensitive-schema`, `/api/sensitive-schema/discover`, `/toggle`, `/bulk-toggle`
- **[API]** Discovery candidates review - `/api/discovery-candidates`, `/api/discovery-candidates/{id}`
- **[ENG]** `continuous_data_discovery`, `sensitive_schema_discovery`

11. SIEM integration

- **[UI]** SIEM configuration - `/siem_configuration`, `/submit_siem_configuration`
- **[UI/API]** IPS dashboard (FortiAnalyzer) - `/siem/ips-dashboard`, `/api/siem/ips-dashboard`
- **[API]** IPS report download / schedule / email - `/api/siem/ips-report/download`, `/schedule`, `/email`
- **[ENG]** Rapid7 + CrowdStrike forwarders (`config.siem`, `siem` schema)

12. Retention & disk management

- **[UI]** Retention policy - `/retention_policy`, `/submit_retention_policy`
- **[API]** Retention policies CRUD + executions - `/api/retention/policies` (GET/POST/PUT), `/api/retention/executions`
- **[JOB]** `dump_and_prune_metrics` - CSV dump + prune by per-metric retention
- **[JOB]** `retention_space_alert` - email when free disk < threshold
- **[ENG]** `retention_engine`, `dump_metrics`

13. Global parameters / configuration

- **[UI/API]** Global params CRUD - `/global_params`, `/global_param_set`, `/submit-global-param`, `/api/global-params`, `/api/delete-global-param`
- Keys: ports, dry-run flags, ingest URL/key, dump location, disk-alert %, retention days, paths

14. GRC - Governance, Risk & Compliance

Dashboards/pages (`/grc/*`) + APIs (`/api/*`):

- **Compliance reporting** - `/grc/compliance-reports`, `/api/compliance-reports`, `/run`, `/schedules` (GET/PUT), `/download/{file}`; **[JOB]** `compliance_reports_daily` (PCI-DSS/HIPAA), `compliance_reports_weekly` (GDPR/SOC2)
- **Audit log** - `/grc/audit-log`, `/api/audit-log`
- **DDL audit** - `/grc/ddl-audit`, `/api/ddl-audit/logs`, `/api/ddl-audit/summary`; **[JOB]** `ddl_audit_scan`
- **Immutable audit hash chain** - **[JOB]** `hash_chain_write` (60s), `hash_chain_verify` (daily) - SHA-256 chain over audit rows
- **Firewall policies** - `/grc/policies`, `/api/firewall-policies` (GET/POST/PUT/DELETE), `/apply-template`; **[JOB]** `grc_firewall_scan`
- **Segregation of Duties** - `/grc/sod-violations`, `/api/sod/rules` (GET/POST/PUT), `/api/sod/violations`, `/acknowledge`; **[JOB]** `sod_violation_scan`
- **Risk register & scoring** - `/grc/risk-dashboard`, `/api/risk-scores`, `/{server}/{login}/events`; **[JOB]** `user_risk_scoring`
- **Control attestation** - **[JOB]** `attestation_scheduler` (`config.attestation_controls`/`periods`)
- **Threat response** - `/grc/threat-response`, `/api/threat-response/playbooks` (GET/POST/PUT), `/blocked-ips` (GET/POST/DELETE), `/suspended-users` (GET/POST/DELETE), `/incidents` (GET/POST/PUT), `/response-log`; **[JOB]** `run_threat_response` (escalate/notify/suspend/block-IP)
- **Cross-border transfer tracking** - `/grc/...`; **[JOB]** `cross_border_scan` (`config.data_regions`/`transfer_agreements`)
- **Access review** - `/grc/access-review`, `/api/access-review/cycles` (GET/POST), `/instances`
- **Incident / GDPR 72h** - `/api/threat-response/incidents`; **[JOB]** `incident_gdpr_timer`
- **Policy exceptions** - `/grc/policy-exceptions`, `/api/policy-exceptions` (GET/POST/PUT)
- **Privilege-change tracking** - `/grc/privilege-changes`, `/api/privilege-changes`, `/recent-summary`; **[ENG]** `privilege_change_scanner`

- **Regulation controls** - `/grc/regulation-controls`, `/api/regulation-controls`
- **Evidence packages** - `/grc/evidence-packages`, `/api/evidence-packages`, `/generate`, `/{id}/download/{type}`; **[ENG]** `evidence_package_generator`
- **TLS enforcement** - `/grc/tls-enforcement`, `/api/tls/policies` (GET/POST/PUT), `/api/tls/violations`; **[ENG]** `tls_enforcement_scanner`
- **Vulnerability assessment** - `/grc/vulnerability-assessment`, `/api/vulnerability/findings`, `/cve-watchlist` (GET/POST/PUT), `/scan`; **[ENG]** `vulnerability_scanner`
- **Data retention (GRC view)** - `/grc/data-retention`
- **Masking coverage** - `/grc/masking-rules`; **[JOB]** `sync_masking_rules`

15. Visualization (Grafana, 91 dashboards)

Discovery & PII, Activity & Audit, Threat/Injection, Policy & Protection, Vulnerability, Automation/Workflows, the 15-dashboard GRC suite, SIEM/IPS, Retention, Blocker, Configuration, and the Israeli PPL dashboard - all over the `dbanalytics` PostgreSQL datasource with `${global_ip}`-templated links.

16. Integrations & packaging

- **Cloud ingest** to `dbexpertai.com/api/ingest` (API key)
- **SIEM**: Rapid7, CrowdStrike; **IPS**: FortiAnalyzer
- **SSRS** report download/attach; **SMTP** email
- **Packaging**: full installer (`dbdome_setup`), incremental updater (`dbdome_update`), Ubuntu appliance (eggs ISO), Windows services via NSSM

---

17. Grafana dashboards - full panel / button / link catalog

Shared dashboard chrome (on essentially every dashboard): a top nav menu linking *Alert dashboard, Alert rules, Alert thresholds, Email configuration, Risk-level alert, Webhook alerts*; and on **every panel** an *Email panel* (`/api/email-panel-form`) and *Print panel* (`/api/print-panel`) action. These are omitted from the per-dashboard lists below.

Totals: **89 dashboards, 895 panels, 74 action buttons**.

1. Data Discovery & Classification

Variables: global*ip, issue*id, rc_id

Panels (27): table x16, stat x5, text x4, timeseries x1, bargauge x1

- Sensitive Columns (PII), PII Issues Detected, PII Access Alerts, Audit Issues, Servers Scanned, Issues (PII), Root causes (PII) - ${issue*id}, Metric metadata - ${rc*id}, Sensitive Data (PII) - Sensitive columns actively queried (PII in use), Alerts, PII Root Cause Findings (SEC-SQL-PRI), Audit Findings Over Time, Top Audit Root Causes, Audit Trail Findings (SEC-SQL-AUD), Sensitive by table , column, Sensitive Transactions by User, Misunderstanding of Encryption (TDE), Performance Overhead Fears, PII Exposed in Clear Text: Legacy Schema Constraints, In-House ""Masking"" Logic Failure, Key Management Complexity, Lack of Native Masking Features (Old Versions), Unsecured Backups/Snapshots

- **Action buttons:** /generate*report/DAM%20Compliance%20Report, /generate*report/Privileged%20Access%20Oversight%20Report, /sensitive_schema
- **Links:** /sensitive_schema
- **Drill-downs:** /d/addpcfp/1-data-discovery-and-classification, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

10. Unknown TCP connections

Variables: global_ip

Panels (4): text x3, table x1

- Unkown TCP connections

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

11. Transaction requests

Variables: global_ip

Panels (4): text x3, table x1

- Trnsaction requests

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

2. Activity Monitoring & Audit

Variables: server, login*name, transaction*status, database*name, global*ip

Panels (12): stat x5, text x3, table x2, timeseries x1, barchart x1

- Active Transactions, Unique Users, Servers, Anomaly Findings, Alerts Sent, Transaction Activity Over Time, Top Users by Transactions, Print (filtered view)

- **Action buttons:** /generate*report/Privileged%20Access%20Oversight%20Report, /generate*report/rpt*2*activity*monitoring*audit*transactions, /sensitive*schema

- **Links:** /sensitive_schema

- **Drill-downs:** /d/adbcsrw/2-activity-monitoring-and-audit, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

2. Schema DML Tracking

Variables: global_ip

Panels (13): stat x5, text x3, table x3, timeseries x1, barchart x1

- Active Transactions, Unique Users, Servers, Anomaly Findings, Alerts Sent, Transaction Activity Over Time, Top Users by Transactions, Transaction Anomaly Findings (SEC-SQL-ACC-010), Sensitive Data Transactions

- **Action buttons:** /generate*report/Privileged%20Access%20Oversight%20Report, /generate*report/rpt*2*activity*monitoring*audit*transactions, /sensitive*schema

- **Links:** /sensitive_schema

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

3. Threat Detection & Behavioral Analytics

Variables: global*ip, root*cause_id

Panels (22): table x10, stat x7, text x3, timeseries x1, bargauge x1

- SQL Injections, Credentials Exposed, Anomaly Findings, Policy Violations, Locked Accounts, Alerts Sent, Active Users, Threat Findings Over Time, Top Threat Root Causes, SQL Injection Types by Root Cause (from rootcause taxonomy), SQL Injection Findings, Credentials Stored in Database Tables, Database Restored, Enabled Sysadmin, Weak Password Enforcement, Linked Server / DBLINKS Hidden Credentials, Scanned Jobs for Leaks, Policy Not Enforced

- **Drill-downs:** /d/adccmtx/3-threat-detection-and-behavioral-analytics, /d/adm7hpj, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

4. Policy Enforcement & Protection

Variables: global*ip, root*cause_id

Panels (10): stat x5, text x3, table x1, barchart x1

- Privileged Logins, Weak Password Policies, Credentials in Tables, Policy Not Enforced, Policy Findings, Root causes (PII) - ${root*cause*id}, Policy Violations by Type

- **Drill-downs:** /d/addpcfp/1-data-discovery-and-classification, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

5. Data Protection

Variables: global*ip, root*cause*id, detail*server, detail*table, detail*column

Panels (12): table x5, stat x4, text x3

- Unmasked Columns, PII Columns, Encryption Findings, Masking Findings, Unmasked Data by Server, Data Protection - Unmasked Columns, Sensitive Schema (PII Columns), Privileged Logins, Root cause details - ${root*cause*id} ${detail*server} ${detail*table} ${detail_column}

- **Links:** /sensitive_schema

- **Drill-downs:** /d/ad88jgn/5-data-protection, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

6. Vulnerability Assessment

Variables: global_ip

Panels (15): stat x5, table x4, text x3, barchart x1, piechart x1, timeseries x1

- Privileged Logins, Weak Passwords, Config Findings, Network Findings, Total Vulnerabilities, Vulnerabilities by Area, By Severity, Vulnerability Findings Over Time, Server Hardening Findings, Vulnerability Root Cause Findings

- **Links:** /sensitive_schema

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

7. Automation & Workflows

Variables: global*ip, root*cause_id

Panels (18): stat x7, table x6, text x3, timeseries x1, barchart x1

- Servers, Root Causes, Detection Rules, Custom Metrics, Active Transactions, Metric Results, Log Entries, Metric Executions Over Time, Enable / Disable Metrics per Server, All Metrics (Latest Execution), Operation Log, Log Entries by Routine, Root cause details - ${root*cause*id}

- **Drill-downs:** /d/adnz9dq, /d/adsddxs/7-automation-and-workflows, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

8. Database restored

Variables: global_ip

Panels (3): text x2, table x1

- Database restored

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

9. Connections per user

Variables: global_ip

Panels (6): text x4, table x2

- Connections per user, Unkown TCP connections

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Alerts

Variables: global*ip, risk*level, root*cause*id, alert_id

Panels (12): stat x6, text x4, table x2

- Total Alerts, Critical, High, Medium, Low alerts, Servers, Alerts Sent by Mail, Active Detection Findings

- **Action buttons:** /generate*report/DAM%20Compliance%20Report, /generate*report/Privileged%20Access%20Oversight%20Report, /report_schedule, /reports

- **Drill-downs:** /d/ad7kkx7/alerts, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Audit expired

Variables: global_ip

Panels (12): text x4, stat x4, table x2, barchart x1, bargauge x1

- Expired Audit Records, Servers Affected, Audit Root Causes, Audit Alerts Sent, Expired Audits by Server, Top Audit Root Causes, Audit Expired Records, Audit Root Cause Findings (SEC-SQL-AUD)

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Blocker Activity

Panels (10): stat x5, text x3, table x2

- Killed, Dry-run, Skipped, Errors, Dry-run mode, Blocker log, Executed blocks (alerts.blocks)

Blocking transactions

Variables: global_ip

Panels (11): barchart x4, text x3, table x2, bargauge x1, stat x1

- blocked transactions, Active users, SQL injection by patterns, Recent alerts - SQL injections, Connectivity reports, Sensitivity reports, SQL Injection analysis

- **Links:** d/adcrpzr/blocking-transactions

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

Blocks in databases

Variables: global_ip

Panels (12): text x5, barchart x2, table x2, bargauge x1, stat x1, gauge x1

- Database Blocks, Active users, Database blocks, Recent alerts, Connection number per user, Active threats

- **Action buttons:** /download*report/block*in*transactions*report, /reportbymail/block*in*transactions_report

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/ad6lv7f/transaction-report, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding ...

Configuration

Variables: global_ip, server

Panels (31): table x12, stat x6, text x6, piechart x3, timeseries x2, barchart x2

- Monitored Servers, Active Detection Rules, Alert Rules, Sensitive Columns, Scheduled Reports, Alerts Sent (7d), Detection Rules by Vendor, Detection Rules by Domain, Alerts Sent Over Time, Alerts by Severity, Configured Reports, Alert Thresholds, Sensitive Schema - PII Columns, PII Columns by Category, Webhook Alert Channels, Global Parameters, Recent Operation Log, Detection Results Over Time, Top Root Causes by Findings, Rootcause Taxonomy Overview, Mail Configuration, Mail Groups & Recipients, Risk Levels, Print (filtered view)

- **Action buttons:** /addrecipients, /generate*report/DAM%20Compliance%20Report, /generate*report/Privileged%20Access%20Oversight%20Report, /mailconfiguration, /report_schedule, /reports, /risklevel
- **Links:** /global*params, /report*schedule, /sensitive*schema, /webhook*alerts
- **Drill-downs:** /d/dbdome-configuration/configuration, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Connectivity

Variables: global_ip

Panels (16): stat x5, text x4, barchart x3, piechart x2, table x2

- Active Connections, Unique Clients, Servers, Network Findings, Connection Findings, Connections by Server & Client, By Protocol, By Auth Scheme, TCP Connections, Network & Connection Root Cause Findings, Data Activity (Transactions/hour), Connections per Client

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Credentials stored in tables

Variables: global_ip

Panels (12): text x4, stat x4, table x2, barchart x1, piechart x1

- Credentials Found, Servers Affected, Tables with Credentials, Root Cause Findings, Credentials by Server & Table, By Server, Credentials Stored in Database Tables, PII/Credential Root Cause Findings

- **Links:** /sensitive_schema
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Custom metrics

Variables: global_ip

Panels (15): stat x5, text x4, piechart x2, table x2, barchart x1, timeseries x1

- Total Metrics, Active Metrics, Inactive Metrics, Metric Results (period), Matched Findings, Metrics by Vendor, By Type, Top Executed Metrics, Custom Metrics Configuration, Metric Results with Comparison, Metric Executions Over Time

- **Links:** /custom_metrics

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Database connections per user

Variables: global_ip

Panels (13): stat x5, text x4, table x2, barchart x1, piechart x1

- Total Connections, Unique Users, Servers, Multi-Host Alerts, Connection Findings, Connections per User, By Server, Connections per User (Detail), Connection Root Cause Findings

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Database restored

Variables: global_ip

Panels (12): text x5, gauge x2, table x2, bargauge x1, stat x1, barchart x1

- Database restored, Active users, Recent alerts, Connection number per user, Active threats

- **Action buttons:** /download*report/database*restored*report, /reportbymail/database*restored_report
- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/ad6lv7f/transaction-report, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding ...

Email configuration

Variables: global_ip

Panels (6): text x4, table x2

- Email configuration, Mail Groups & Recipients

- **Action buttons:** /addrecipients, /mailconfiguration
- **Links:** /email_configuration
- **Drill-downs:** /d/adcdjsj/email-configuration, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Enabled sysadmin

Variables: global_ip

Panels (12): text x5, barchart x2, table x2, bargauge x1, stat x1, gauge x1

- Enabled sysadmin, Active users, Recent alerts, Connection number per user, Active threats

- **Action buttons:** /download*report/enabled*sysadmin*report, /reportbymail/enabled*sysadmin_report
- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adnskjz/enabled-sysadmin, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding ...

GRC Access Review

Variables: global_ip

Panels (13): stat x4, table x3, text x2, barchart x2, timeseries x1, piechart x1

- Open Review Instances, Overdue Reviews, Completed (90d), Active Cycles, Reviews Opened vs Completed per Month, Instances by Status, Open Reviews by Cycle, Avg Days to Complete by Cycle, Overdue Review Instances - Action Required, All Open Review Instances, Review Cycle Configuration

GRC Audit Log

Variables: global*ip, server*filter

Panels (10): stat x3, barchart x3, text x2, timeseries x1, table x1

- ALERTED Events, BLOCKED Events, MASKED Events, Events per Hour by Action, Top Source IPs (BLOCKED), Top Users (BLOCKED), Top Servers (all actions), Audit Events

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

GRC Compliance

Variables: global_ip

Panels (11): stat x4, table x3, text x2, timeseries x1, barchart x1

- PCI-DSS Events, HIPAA Events, GDPR Events, SOC2 Events, Events by Regulation (hourly), Actions by Regulation, Regulation Breach Events (BLOCKED / ALERTED), Masking Coverage by Regulation, Daily Audit Summary by Action

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

GRC Control Attestation Workflow

Variables: global_ip

Panels (15): stat x4, gauge x4, text x2, table x2, timeseries x1, piechart x1, barchart x1

- Pending Attestations, Exception Attestations, Open Periods, Completion Rate (Latest Period), Attestations Completed per Month, Attestation Status Distribution, Pending by Regulation, Completion Rate - SOC2, Completion Rate - SOX, Completion Rate - PCI-DSS, Completion Rate - GDPR, Pending Attestations - Action Required, Attestation Exceptions

GRC Cross-Border Transfer Tracking

Variables: global_ip

Panels (12): stat x4, table x3, text x2, timeseries x1, piechart x1, barchart x1

- Transfers Detected (30d), Without Legal Basis (30d), Critical Risk Transfers, Server Regions Configured, Cross-Border Transfers per Day, Transfer Mechanisms Used, Transfers by Destination Country (30d), Transfer Agreements, Transfers Without Legal Basis - Action Required, Server Region Registry

GRC Immutable Audit Hash Chain

Variables: global_ip

Panels (10): stat x4, text x2, table x2, timeseries x1, barchart x1

- Total Rows Hashed, Rows Pending Hash, Last Verification Result, Verifications Run, Recent Audit Log - Hash Status, Rows Hashed per Hour, Verification Results (Last 30), Hash Chain Verification Checkpoints

GRC Incident Lifecycle Management

Variables: global_ip

Panels (13): stat x4, table x3, barchart x2, text x2, timeseries x1, piechart x1

- Active Incidents, Critical Active, GDPR 72h Overdue, CAPA Actions Open, Incidents per Week by Severity, Incidents by State, Incidents by Category (90d), Avg Resolution Time by Severity (h), GDPR Breach Notifications, Open CAPA Actions

GRC Masking Coverage

Variables: global_ip

Panels (12): stat x4, table x3, piechart x2, text x2, barchart x1

- Active Masking Rules, Sensitive Columns (total), Unmasked Sensitive Columns, Token Vault Entries, Rules by Mask Type, Rules by PII Type, Active Rules by Regulation, Unmasked Sensitive Columns (Risk Gap), Masking Coverage Summary

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

GRC Overview

Variables: global_ip

Panels (21): stat x13, text x3, row x2, barchart x1, timeseries x1, table x1

- BLOCKED (24h), ALERTED (24h), Open Incidents, High-Risk Users (≥70), Active Blocked IPs, Active Masking Rules, Top Servers by Events (24h), Firewall Events by Action (hourly), Recent BLOCKED / CRITICAL Events, Phase 8 KPIs, Exceptions Pending Approval, Overdue Access Reviews, Phase 4 KPIs, Open Critical Incidents, GDPR 72h Overdue, Pending Attestations, Transfers Without Legal Basis, Hash Chain Valid

- **Action buttons:** /d/grc-access-review, /d/grc-attestations, /d/grc-audit-log, /d/grc-compliance, /d/grc-cross-border, /d/grc-ddl-privileges, /d/grc-discovery, /d/grc-evidence, /d/grc-hash-chain, /d/grc-incidents, /d/grc-masking, /d/grc-overview, /d/grc-policy-exceptions, /d/grc-retention, /d/grc-risk-register, /d/grc-risk-scoring, /d/grc-sod, /d/grc-threat-response, /d/grc-tls-vuln, /d/grc-workflow-sod
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

GRC Policy Exceptions

Variables: global_ip

Panels (14): stat x4, barchart x3, table x3, text x2, timeseries x1, piechart x1

- Pending Approval, Active Approved Exceptions, Expiring in <7 Days, Rejected (30d), Exception Requests per Day, Exceptions by Status, Active Exceptions by Regulation, Exceptions by Server, Top Requestors (90d), Pending Approval - Action Required, Exception History (All)

GRC Risk Register

Variables: global_ip

Panels (12): stat x4, barchart x4, text x2, piechart x1, table x1

- Open Risks, Critical / High Risks (Score ≥15), In Treatment, Avg Residual Risk Score, Risks by Inherent Score Bucket, Risks by Category, Risks by Treatment Strategy, Top Assets by Residual Risk Score, Risks by Regulation, Risk Register

GRC Risk Scoring

Variables: global_ip

Panels (10): stat x3, text x2, table x2, gauge x1, timeseries x1, barchart x1

- High-Risk Users (≥70), Critical-Risk Users (≥90), Average Risk Score, Monitored Users, Risk Score Trend (events), Users by Risk Band, Top 30 High-Risk Users, Recent Risk Events

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

GRC Threat Response

Variables: global_ip

Panels (13): table x5, stat x4, text x2, gauge x1, timeseries x1

- Response Success Rate (%), Open Incidents, Blocked IPs (active), Suspended Users (active), Responses (24h), Active Suspended Users, Automated Responses per Hour, Open Security Incidents, Recent Automated Responses, Active Blocked IPs, Active Response Playbooks

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

GRC Workflow Segregation of Duties

Variables: global_ip

Panels (11): stat x4, barchart x2, table x2, text x2, timeseries x1

- SoD Violations (30d), Violations This Week, Active SoD Rules, Unique Violators (30d), Workflow SoD Violations per Day, Violations by Action Type (30d), Top Violators (30d), SoD Rule Configuration, Workflow SoD Violation Log (30d)

IPS Dashboard - FortiAnalyzer

Variables: global_ip

Panels (15): stat x6, table x4, text x2, piechart x1, barchart x1, timeseries x1

- Total Events, Critical, High, Medium, Blocked, Monitored, Intrusions by Severity, Intrusions by Type, Intrusion Events Timeline (Last 7 Days), Monitored Intrusions, Top Victims, Blocked Intrusions, Top Attack Sources

- **Links:** /api/siem/ips-report/download, /siem/ips-dashboard

Linked servers / DBLINKS - Hidden credentials

Variables: global_ip

Panels (12): text x5, gauge x2, table x2, bargauge x1, stat x1, barchart x1

- Linked server / DBLINKS Hidden credentials, Active users, Recent alerts, Connection number per user, Active threats

- **Action buttons:** /download*report/linked*server*report, /reportbymail/linked*server_report

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/ad6lv7f/transaction-report, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding ...

Locked accounts

Variables: global_ip

Panels (11): text x5, barchart x4, table x2

- Locked accounts, Sensitivity reports, SQL Injection analysis, Data activity monitoring

- **Action buttons:** /download*report/locked*accounts*report, /reportbymail/locked*accounts_report

- **Drill-downs:** /d/ad8pvv6/locked-accounts, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Mail

Panels (3): text x2, table x1

- Mail configuration

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

New dashboard

Panels (3): text x2, timeseries x1

- report name

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

PII - Sensitive columns

Variables: global*ip, root*cause_id

Panels (11): stat x6, table x3, text x2

- Total Events, Critical, High, Medium, Blcked, Monitored, Root causes, Detailed findings, Root cause details - ${root*cause*id}

- **Links:** /api/siem/ips-report/download, /siem/ips-dashboard
- **Drill-downs:** /d/adw9lx6/pii-sensitive-columns

PII - Sensitive data in use

Variables: global*ip, root*cause_id

Panels (11): stat x6, table x3, text x2

- Total Events, Critical, High, Medium, Blcked, Monitored, Root causes, Detailed findings, Root cause details - ${root*cause*id}

- **Links:** /api/siem/ips-report/download, /siem/ips-dashboard
- **Drill-downs:** /d/ad8885r/pii-sensitive-data-in-use

PPL- Protection of Privacy Law, 5741 - 1981

Variables: global_ip

Panels (16): stat x6, table x5, text x2, piechart x1, barchart x1, timeseries x1

- Total Events, Critical, High, Medium, Blcked, Monitored, Intrusions by Severity, Intrusions by Type, Intrusion Events Timeline (Last 7 Days), Blocked Intrusions, Monitored Intrusions, Top Victims, Root causes, Top Attack Sources

- **Links:** /api/siem/ips-report/download, /siem/ips-dashboard

Policy not enforced

Variables: global_ip

Panels (11): text x5, barchart x4, table x2

- Policy not enforced, Sensitivity reports, SQL Injection analysis, Data activity monitoring

- **Action buttons:** /download*report/policy*not*enforeced*report, /reportbymail/policy*not*enforeced_report
- **Drill-downs:** /d/ad99j9q/policy-not-enforced, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Processes

Variables: global_ip

Panels (10): text x5, barchart x4, table x1

- Processes, Sensitivity reports, SQL Injection analysis, Data activity monitoring, Connectivity reports

- **Links:** /processform
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Recent alerts

Variables: global_ip

Panels (15): table x11, text x4

- Security dashboard, Critical issues, Active threats, Active users, Cyber attacks, Recent alerts, Sensitivity reports, SQL Injection analysis, Data activity monitoring, Connectivity reports

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Retention

Variables: dump*location, edit*metric, edit*value, dump*metric, restore_id

Panels (10): table x8, text x2

- Retention Overview, Apply retention edit (auto-runs on / click), Current dump location, Save dump location (refresh to save), Dump metrics (config.dump_metrics), Dumps catalog (config.dumps), Apply dump (auto-runs on Dump click), Apply restore (auto-runs on Restore click)

- **Drill-downs:** /d/adretn001/retention

Retention policy

Variables: global_ip

Panels (9): text x4, barchart x4, table x1

- Retention policy, Sensitivity reports, SQL Injection analysis, Data activity monitoring, Connectivity reports

- **Links:** /retention_policy
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Rootcauses

Variables: sel*issue, global*ip

Panels (4): text x2, table x2

- Issues (click 'select' to load its root causes), Root causes for selected issue - toggle active / set risk level (saved via API)

- **Links:** /root*cause*active, /root*cause*risk
- **Drill-downs:** /d/rootcauses/rootcauses

SIEM configuration

Variables: global_ip

Panels (8): barchart x4, text x3, table x1

- SIEM, Sensitivity reports, SQL Injection analysis, Data activity monitoring, Connectivity reports

- **Links:** /siem*configuration, http://${global*ip}:8080/siem_configuration
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

SQL Injection

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- SQL injection by patterns, SQL Injections - Injection prevention

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Boolean-based injections

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, comment based injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Comment based

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, comment based injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Information schema probing

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Information schema probing

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Stacked queries

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Stacked query injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Time based

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Stacked query injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Time based blind injection

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Time based blind injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - Union based

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Union based injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection - tautology with comments

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, comment based injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection -Authentication Bypass

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Authentication bypass

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection -Encoding

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Encoding injection

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection -Error based injection

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Error based injections

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection -Shell function

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Shell commands

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection -order / Group by

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, Order / Group by

- **Drill-downs:** /d/ad47gg4/sql-injection-shell-function, /d/ad4j4rm/sql-injection-error-based-injection, /d/adcqwvk/sql-injection-time-based-blind-injection, /d/adk5ltv/sql-injection-tautology-with-comments, /d/adkk925/sql-injection-boolean-based-injections, /d/adltjt8/sql-injection-order-group-by, /d/admk64d/sql-injection-authentication-bypass, /d/adn2bdh/sql-injection-information-schema-probing, /d/adp58pz/sql-injection-union-based, /d/adphgv5/sql-injection-comment-based, /d/adq45cm/sql-injection-encoding, /d/adw44xx/sql-injection-stacked-queries ...

SQL injection board

Variables: global_ip

Panels (6): text x4, gauge x1, table x1

- Critical issues - SQL injection, SQL injection - Sleep time

- **Drill-downs:** /d/ad4btvb/sql-injection-sleep-time, /d/adddd87/sql-injection-boolean-usage, /d/adlpnn2/sql-injection-sensitive-data, /d/adp5zsq/sql-injection-outfile-copy, /d/adqsd5f/sql-injection-union-usage, /d/adrlq5l/sql-injection-dangerous-queries, /d/adwr6tl/sql-injection-stacked-query, /d/adzr5nq/sql-injection-long-literal, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview ...

Same login active from multiple hosts

Variables: global*ip, root*cause_id

Panels (5): text x3, table x2

- Root causes, Detailed findings

- **Links:** /api/siem/ips-report/download, /siem/ips-dashboard
- **Drill-downs:** /d/adlvvwg/same-login-active-from-multiple-hosts

Scan jobs for leaks

Variables: global_ip

Panels (13): text x4, stat x4, barchart x3, bargauge x1, table x1

- Jobs with Leaks, Servers Affected, Security Findings, Alerts Sent, Leaked Jobs by Server, Sensitivity Issues by Server, Scanned Jobs for Leaks, SQL Injection Analysis, Data Activity (Transactions/hour)

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Security Issues Explorer

Variables: global_ip, rc

Panels (4): text x2, table x2

    - Security hierarchy - click a root cause to filter alerts, Alerts for selected root cause - ${rc}
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response, /d/sec-explorer/security-issues-explorer

Send report by mail

Variables: global_ip

- **Panels (7):** text x4, table x3
- **Action buttons:** /download*report/transaction*report, /reportbymail/transaction_report
- **Drill-downs:** /d/ad6lv7f/transaction-report, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Sensitive Columns Explorer

Variables: srv, db, global_ip, dsch, dtbl, dcol

Panels (13): table x10, text x3

- Servers (sensitive columns), Databases on ${srv}, Sensitive columns - ${srv} / ${db}, Column details - ${dtbl}.${dcol}, Tracked sensitive columns (metrics.sensitive*columns), Masking activity (metrics.masking*log), Masked data preview - real vs masked (as test user), Print (filtered view), Encryption activity (metrics.encryption_log), Tokenized data preview - original vs token

- **Links:** /encrypt*column, /mask*column, /revert*encryption, /sensitive*column_add
- **Drill-downs:** /d/adpiicol/sensitive-columns-explorer

Sensitivity report

Variables: global_ip

Panels (10): text x5, table x5

- Sensitive data (PII), Sensitive columns and schema (PII)

- **Action buttons:** /download*report/sensitive*schema*report, /reportbymail/sensitive*schema_report
- **Drill-downs:** /d/adgp5hv/sensitivity-report, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Stored procedure slower than its average

Variables: global*ip, root*cause_id

Panels (5): table x3, text x2

- Root causes, Detailed findings, Root cause details - ${root*cause*id}

- **Links:** /api/siem/ips-report/download, /siem/ips-dashboard
- **Drill-downs:** /d/ad55bbz/stored-procedure-slower-than-its-average

Super users (sysadmin / sysDBA)

Variables: global_ip

Panels (10): text x5, table x2, barchart x1, bargauge x1, stat x1

- SYSADMIN / SYSDBA, Active users, Recent alerts, Credentials stored in database tables

- **Action buttons:** /download*report/sysadmin*report, /reportbymail/sysadmin_report
- **Drill-downs:** /d/adbzzdg/super-users-sysadmin-sysdba, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Take Action

Variables: global_ip

Panels (14): text x5, barchart x4, gauge x2, bargauge x1, table x1, stat x1

- Critical issues - SQL injection, Active threats, Active users, Take action, Recent alerts, Sensitivity reports, SQL Injection analysis, Data activity monitoring, Connectivity reports

- **Links:** /take_action
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Transaction report

Variables: global_ip

Panels (15): text x7, table x4, barchart x4

- Transactions, Sensitivity reports, SQL Injection analysis, Data activity monitoring, Connectivity reports

- **Action buttons:** /download*report/blocking*transaction*report, /reportbymail/blocking*transaction_report
- **Drill-downs:** /d/ad6lv7f/transaction-report, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Vulnerability

Variables: global_ip

Panels (14): stat x5, text x4, barchart x3, piechart x1, table x1

- SQL Injections, Suspicious Queries, Config Vulnerabilities, Active Users, Total Findings, Vulnerability by Issue Type, By Security Area, Security Vulnerability Findings, SQL Injection by Server, Connectivity by Server

- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

Webhook Alerts Configuration

Variables: global_ip

Panels (6): text x3, table x3

- Webhook Alerts - click a true/false cell to toggle, Blocker global dry-run (true = log only, false = ARMED to kill), Masking global dry-run (true = log only, false = ARMED to apply masks)

- **Links:** /global*param*set, /webook*alert*set

alert_report

Panels (4): text x2, nodeGraph x1, table x1

- Open Alerts

- **Links:** http://${global*ip}:8080/open*alert*change*astaus
- **Drill-downs:** /d/${*_data.fields.report*url}, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

alert*report*Error_Based

Panels (4): text x2, nodeGraph x1, table x1

- Open Alerts

- **Links:** http://${global*ip}:8080/open*alert*change*astaus

- **Drill-downs:** /d/${_*data.fields.report*url}, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

alert*report*Privilege*Escalation*Chain

Panels (4): text x2, nodeGraph x1, table x1

- Open Alerts

- **Links:** http://${global*ip}:8080/open*alert*change*astaus
- **Drill-downs:** /d/${_*data.fields.report*url}, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

alert*report*Stacked

Panels (4): text x2, nodeGraph x1, table x1

- Open Alerts

- **Links:** http://${global*ip}:8080/open*alert*change*astaus
- **Drill-downs:** /d/${_*data.fields.report*url}, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

alert*report*sensitive_schema

Panels (5): table x2, text x2, nodeGraph x1

- Open Alerts, Sensitive columns and schema (PII), Sensitive data (PII)

- **Drill-downs:** /d/${_*data.fields.report*url}, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

risk level alerts

Variables: global_ip

Panels (6): text x4, table x2

- Risk level alerts, Risk Levels

- **Action buttons:** /risklevel, /risklevel_toggle
- **Links:** /email_configuration
- **Drill-downs:** /d/ad54967/risk-level-alerts, /d/adcdjsj/email-configuration, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

sysadmin accounts with weak password enforcement

Variables: global_ip

Panels (12): text x6, barchart x4, table x2

- sysadmin accounts with weak password enforcement, Sensitivity reports, SQL Injection analysis, Data activity monitoring

- **Action buttons:** /download*report/week*passwords, /reportbymail/transaction_report
- **Drill-downs:** /d/admz7fh/sysadmin-accounts-with-weak-password-enforcement, /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

webhook alerts

Variables: global_ip

Panels (4): text x3, table x1

- web hook alerts

- **Links:** /webhook_alerts
- **Drill-downs:** /d/grc-audit-log, /d/grc-compliance, /d/grc-masking, /d/grc-overview, /d/grc-risk-scoring, /d/grc-threat-response

---

18. Configuration menu (admin UI nav -> page -> route)

The admin **Configuration** menu and the page each item opens:

Menu item	Page title	Route(s)	Catalog §
Servers	Configure Servers	`/serverform`, `/submit-serverform`, `/server_enable`	§2
Processes	Configure Processes	`/processform`, `/submit-processform`	§5
Custom Metrics	Custom Metrics	`/custom_metrics`, `/submit_custom_metrics`	§3
Mail Config	SMTP Configuration	`/mailconfiguration`, `/submit-mailconfiguration`, `/addrecipients`	§7
SIEM	SIEM Configuration	`/siem_configuration`, `/submit_siem_configuration`	§11
Report Scheduling	Schedule Reports	`/report_schedule`, `/submit-report-schedule`	§8
Sensitive Schema	PII Discovery	`/sensitive_schema`, `/sensitive_column_add`, `/api/sensitive-schema/discover`	§10
Alert Thresholds	Configure Thresholds	`/alert_thresholds`, `/submit-alert-threshold`	§6
Alert Rules	Configure Alert Rules	`/alert_rules`, `/submit-alert-rule`	§6
Global Parameters	System Settings	`/global_params`, `/global_param_set`, `/submit-global-param`	§13
Webhook Alerts	Alert Channels	`/webhook_alerts`, `/webook_alert_set`, `/submit-webhook-alert`	§6
Alert Dashboard	Security Alerts	`/alert_dashboard`, `/api/alert_dashboard`	§6
Retention Policy	Data Retention	`/retention_policy`, `/submit_retention_policy`	§12
Report Definitions	Edit Report JSON	`/reports`, `/reports/edit`, `/reports/save`	§8

All 14 are backed by FastAPI routes in `http_server.py` and persist to the `config` / `metrics` schemas (see §10 of the spec for the config-table mapping).

Appendix A - Per-panel SQL / query reference

> The SQL behind every Grafana panel (PostgreSQL `dbanalytics` datasource), grouped by dashboard then panel. `${global_ip}`, `${issue_id}`, `${rc_id}` etc. are Grafana template variables resolved at view time. Long lines are wrapped for print. Generated 2026-06-30.

Coverage: **88 dashboards, 600 queries**.

1. Data Discovery & Classification

Sensitive Columns (PII) - *stat*

```
SELECT COUNT(*) FROM alerts.alert_log where root_cause_id = 'SEC-SQL-PRI-001-RC12'
```

PII Issues Detected - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-PRI%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

PII Access Alerts - *stat*

```
SELECT COUNT(*) FROM alerts.mail_alert_log WHERE metric_name LIKE '%%RC05%%' AND $__timeFilter(entry_date)
```

Audit Issues - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-AUD%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Servers Scanned - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.general_metric_metadata_results WHERE (metric_name LIKE 'SEC-SQL-PRI%%' OR
metric_name LIKE 'SEC-SQL-AUD%%') AND $__timeFilter(entry_date)
```

Issues (PII) - *table*

```
select issue_ID , issue_name from ( select distinct issue_ID , issue_name , metric_name from rootcause.v_rootcauses rc left
outer join monitoring.general_metric_metadata_results gmmr on gmmr.metric_name = rc.root_cause_id where rc.domain_code = 'SEC'
and rc.vendor_name = 'sqlserver' ) order by (case when metric_name is null then 0 else 1 end ) desc,metric_name
```

Root causes (PII) - ${issue_id} - *table*

```
select distinct root_cause_id, root_cause_name, root_cause_desc from rootcause.v_rootcauses where domain_code = 'SEC' and
vendor_name = 'sqlserver' and ('${issue_id}' = '' OR issue_id = '${issue_id}')
```

Metric metadata - ${rc_id} - *table*

```
SELECT r.entry_date AT TIME ZONE 'Asia/Jerusalem' AS entry_date, r.server, ( SELECT jsonb_object_agg( k, CASE WHEN
jsonb_typeof(v) = 'number' AND (v::text)::numeric BETWEEN 1000000000000 AND 9999999999999 THEN to_jsonb( to_char(
to_timestamp(((v::text)::numeric) / 1000) AT TIME ZONE 'Asia/Jerusalem', 'YYYY-MM-DD HH24:MI:SS' ) ) ELSE v END ) FROM
jsonb_each(elem) AS j(k, v) ) AS metadata FROM monitoring.general_metric_metadata_results r, LATERAL jsonb_array_elements(
CASE WHEN jsonb_typeof(r.metric_metadata) = 'array' THEN r.metric_metadata ELSE jsonb_build_array(r.metric_metadata) END ) AS
elem WHERE r.metric_name = '${rc_id}' ORDER BY r.entry_date DESC LIMIT 500
```

Sensitive Data (PII) - Sensitive columns actively queried (PII in use) - *table*

```
select * from monitoring.v_sec_sql_pri_001_rc13
```

Alerts - *table*

```
select al.* from alerts.alert_log al left outer join alerts.mail_alert_log mal on mal.server = al.server and mal.metric_name = al.root_cause_id and mal.entry_date = mal.entry_date where root_cause_id = '${rc_id}'
```

PII Root Cause Findings (SEC-SQL-PRI) - *table*

```
SELECT gm.server, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS root_cause,
(gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows_found,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity,
(gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS finding, gm.entry_date AT TIME ZONE 'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name WHERE gm.metric_name LIKE 'SEC-SQL-PRI%%' AND gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date DESC LIMIT 50
```

Audit Findings Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', gm.entry_date) AS time, COALESCE(i.name, gm.metric_name) AS metric, COUNT(*) AS value FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id WHERE gm.metric_name LIKE 'SEC-SQL-AUD%%' AND gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY time, metric ORDER BY time
```

Top Audit Root Causes - *bargauge*

```
SELECT COALESCE(rc.name, gm.metric_name) AS type, COUNT(*) AS count FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name WHERE gm.metric_name LIKE 'SEC-SQL-AUD%%' AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY type ORDER BY count DESC LIMIT 10
```

Audit Trail Findings (SEC-SQL-AUD) - *table*

```
SELECT gm.server, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS root_cause, COALESCE(i.name, '') AS issue, (gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity, gm.entry_date AT TIME ZONE 'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id WHERE gm.metric_name LIKE 'SEC-SQL-AUD%%' AND gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date DESC LIMIT 50
```

Sensitive by table , column - *table*

```
select count(*) "sensitive transactions" , server , table_name , column_name from monitoring.v_sec_sql_PRI_001_RC13 group by server , table_name , column_name
```

Sensitive Transactions by User - *table*

```
select * from monitoring.v_sec_sql_PRI_001_RC13
```

Misunderstanding of Encryption (TDE) - *table*

```
select * from monitoring.v_SEC_SQL_PRI_001_RC01
```

Performance Overhead Fears - *table*

```
select * from monitoring.v_SEC_SQL_PRI_001_RC03
```

PII Exposed in Clear Text: Legacy Schema Constraints - *table*

```
select * from monitoring.v_SEC_SQL_PRI_001_RC05
```

In-House ""Masking"" Logic Failure - *table*

```
select * from monitoring.v_sec_sql_pri_001_rc07 where $__timeFilter(entry_date)
```

Key Management Complexity - *table*

```
select * from monitoring.v_sec_sql_pri_001_rc09 where $__timeFilter(entry_date)
```

Lack of Native Masking Features (Old Versions) - *table*

```
select * from monitoring.v_sec_sql_pri_001_rc10 where $__timeFilter(entry_date)
```

Unsecured Backups/Snapshots - *table*

```
select * from monitoring.v_sec_sql_pri_001_rc11 where $__timeFilter(entry_date)
```

10. Unknown TCP connections

Unkown TCP connections - *table*

```
select * from monitoring.tcp_connections where $__timeFilter(connect_time) and client_net_address not in ( select
client_net_address from monitoring.tcp_connections where connect_time < current_date - interval '1 day' )
```

11. Transaction requests

Trnsaction requests - *table*

```
select * from monitoring.transaction_requests where login_name != 'dbdome_mon_usr' AND $__timeFilter(LAST_request_end_time)
```

2. Activity Monitoring & Audit

Active Transactions - *stat*

```
select count(*) from ( SELECT server, session_id, login_name, host_name, program_name, database_name, transaction_state AS
status, transaction_begin_time at time zone current_setting('TimeZone') AS start_time, NULL::text AS command, NULL::text AS
cpu_time, duration_seconds AS duration, transaction_id, NULL::text AS transaction_name, query_text AS query, NULL::uuid AS
server_id FROM monitoring.v_sec_sql_acc_011_rc02 where entry_date >= $__timeFrom() at time zone current_setting('TimeZone')
and entry_date <= $__timeTo() at time zone current_setting('TimeZone') )
```

Unique Users - *stat*

```
SELECT COUNT(DISTINCT login_name) FROM monitoring.v_sec_sql_acc_011_rc02 WHERE $__timeFilter(entry_date) at time zone
current_setting('TimeZone')
```

Servers - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.v_sec_sql_acc_011_rc02 where entry_date >= $__timeFrom() at time zone
current_setting('TimeZone') and entry_date <= $__timeTo() at time zone current_setting('TimeZone')
```

Anomaly Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-ACC-010%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date) and $__timeFilter(entry_date) at
time zone current_setting('TimeZone')
```

Alerts Sent - *stat*

```
SELECT COUNT(*) FROM alerts.mail_alert_log WHERE $__timeFilter(entry_date)
```

Active Transactions - *table*

```
select * from ( SELECT server, session_id, login_name, host_name, program_name, database_name, transaction_state AS status,
transaction_begin_time at time zone current_setting('TimeZone') AS start_time, NULL::text AS command, NULL::text AS cpu_time,
duration_seconds AS duration, transaction_id, NULL::text AS transaction_name, query_text AS query, NULL::uuid AS server_id ,
```

```
entry_date FROM monitoring.v_sec_sql_acc_011_rc02 ) where entry_date >= $__timeFrom() at time zone current_setting('TimeZone')
and entry_date <= $__timeTo() at time zone current_setting('TimeZone')
```

Transaction Activity Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', a.entry_date) AS time, COUNT(*) AS transactions FROM ( select * from
monitoring.v_sec_sql_acc_011_rc02 where entry_date >= $__timeFrom() at time zone current_setting('TimeZone') and entry_date <=
$__timeTo() at time zone current_setting('TimeZone') )a group by DATE_TRUNC('hour', a.entry_date)
```

Top Users by Transactions - *barchart*

```
SELECT login_name, COUNT(*) AS transactions FROM monitoring.v_sec_sql_acc_011_rc02 where entry_date >= $__timeFrom() at time
zone current_setting('TimeZone') and entry_date <= $__timeTo() at time zone current_setting('TimeZone') group by login_name
```

Print (filtered view) - *table*

```
SELECT ' Print this filtered view (opens clean tab -> Ctrl+P / Save as PDF)' AS print
```

2. Schema DML Tracking

Active Transactions - *stat*

```
SELECT COUNT(*) FROM monitoring.v_perf_sql_tx_010_rc01 WHERE $__timeFilter(entry_date)
```

Unique Users - *stat*

```
SELECT COUNT(DISTINCT login_name) FROM monitoring.v_active_transactions WHERE $__timeFilter(entry_date)
```

Servers - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.v_active_transactions WHERE $__timeFilter(entry_date)
```

Anomaly Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-ACC-010%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Alerts Sent - *stat*

```
SELECT COUNT(*) FROM alerts.mail_alert_log WHERE $__timeFilter(entry_date)
```

Active Transactions - *table*

```
select * from monitoring.v_Active_transactions where $__timeFilter(entry_date)
```

Transaction Activity Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', entry_date) AS time, COUNT(*) AS transactions FROM monitoring.v_PERF_SQL_TX_010_RC01 WHERE
$__timeFilter(entry_date) GROUP BY time ORDER BY time
```

Top Users by Transactions - *barchart*

```
SELECT login_name, COUNT(*) AS transactions FROM monitoring.active_transactions WHERE $__timeFilter(last_request_end_time)
GROUP BY login_name ORDER BY transactions DESC LIMIT 10
```

Transaction Anomaly Findings (SEC-SQL-ACC-010) - *table*

```
SELECT gm.server, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS root_cause,
(gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity,
```

```
(gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE
'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON
rc.root_cause_id = gm.metric_name WHERE gm.metric_name LIKE 'SEC-SQL-ACC-010%%' AND gm.metric_metadata_vs_expected IS NOT NULL
AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date
DESC LIMIT 50
```

Sensitive Data Transactions - *table*

```
select * from monitoring.general_metric_metadata_results where metric_name like 'SEC-SQL-AU-010%'
```

3. Threat Detection & Behavioral Analytics

SQL Injections - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-INJ%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Credentials Exposed - *stat*

```
SELECT COUNT(*) FROM monitoring.v_mssql_credentials_stored_in_tables
```

Anomaly Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-ACC-010%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Policy Violations - *stat*

```
SELECT COUNT(*) FROM monitoring.v_policy_enforeced
```

Locked Accounts - *stat*

```
SELECT COUNT(*) FROM monitoring.v_oracle_locked_accounts
```

Alerts Sent - *stat*

```
SELECT COUNT(*) FROM alerts.mail_alert_log WHERE $__timeFilter(entry_date)
```

Active Users - *stat*

```
SELECT COUNT(*) FROM monitoring.tcp_connections WHERE $__timeFilter(connect_time)
```

Threat Findings Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', entry_date) AS time, COALESCE(i.name, gm.metric_name) AS metric, COUNT(*) AS value FROM
monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT
JOIN rootcause.issues i ON i.issue_id = rc.issue_id WHERE (gm.metric_name LIKE 'SEC-SQL-INJ%%' OR gm.metric_name LIKE
'SEC-SQL-ACC-010%%') AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date)
GROUP BY time, metric ORDER BY time
```

Top Threat Root Causes - *bargauge*

```
SELECT COALESCE(rc.name, gm.metric_name) AS threat, COUNT(*) AS count FROM monitoring.general_metric_metadata_results gm LEFT
JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name WHERE (gm.metric_name LIKE 'SEC-SQL-INJ%%' OR
gm.metric_name LIKE 'SEC-SQL-ACC-010%%') AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND
$__timeFilter(gm.entry_date) GROUP BY threat ORDER BY count DESC LIMIT 12
```

SQL Injection Types by Root Cause (from rootcause taxonomy) - *table*

```
SELECT DISTINCT rc.root_cause_id, rc.root_cause_name AS injection_type, COALESCE(g.finding_count, 0) AS findings,
COALESCE(g.last_detected AT TIME ZONE 'Asia/Jerusalem', NULL) AS last_detected FROM rootcause.v_rootcauses rc LEFT JOIN (
```

```
SELECT metric_name, COUNT(*) AS finding_count, MAX(entry_date) AS last_detected FROM
monitoring.general_metric_metadata_results WHERE $__timeFilter(entry_date) GROUP BY metric_name ) g ON g.metric_name =
rc.root_cause_id WHERE rc.domain_code = 'SEC' AND rc.area_code = 'INJ' ORDER BY findings DESC
```

SQL Injection Findings - *table*

```
select rc.root_cause_name "inection type", (select count(*) from monitoring.general_metric_metadata_results where metric_name
= rc.root_cause_id and entry_date > current_date ) from rootcause.v_rootcauses rc where domain_code = 'SEC' and area_code =
'INJ'
```

Credentials Stored in Database Tables - *table*

```
SELECT * FROM monitoring.v_mssql_credentials_stored_in_tables
```

Database Restored - *table*

```
SELECT * FROM monitoring.v_database_restored
```

Enabled Sysadmin - *table*

```
SELECT * FROM monitoring.v_enabled_sysadmin WHERE $__timeFilter(entry_date)
```

Weak Password Enforcement - *table*

```
SELECT * FROM monitoring.v_sysadmin_accounts_with_weakpassword_enforcement
```

Linked Server / DBLINKS Hidden Credentials - *table*

```
SELECT * FROM monitoring.linked_servers_hidden_credentials
```

Locked Accounts - *table*

```
SELECT server, username, account_status, to_timestamp(lock_date::bigint / 1000) AS lock_date FROM
monitoring.v_oracle_locked_accounts
```

Scanned Jobs for Leaks - *table*

```
SELECT * FROM monitoring.v_scan_jobs_for_leak
```

Policy Not Enforced - *table*

```
SELECT * FROM monitoring.v_policy_enforeced
```

4. Policy Enforcement & Protection

Privileged Logins - *stat*

```
SELECT COUNT(*) FROM monitoring.v_enabled_sysadmin WHERE $__timeFilter(entry_date)
```

Weak Password Policies - *stat*

```
SELECT COUNT(*) FROM monitoring.v_sysadmin_accounts_with_weakpassword_enforcement
```

Credentials in Tables - *stat*

```
SELECT COUNT(*) FROM monitoring.v_mssql_superuser WHERE login_name != 'infosecuser'
```

Policy Not Enforced - *stat*

```
SELECT COUNT(*) FROM monitoring.v_policy_enforeced
```

Policy Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE (metric_name LIKE 'SEC-SQL-AZ%%' OR metric_name LIKE
'SEC-SQL-CFG%%' OR metric_name LIKE 'SEC-SQL-AU%%') AND (metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND
$__timeFilter(entry_date)
```

Root causes (PII) - ${root*cause*id} - *table*

```
select distinct root_cause_id, root_cause_name, root_cause_desc from rootcause.v_rootcauses where domain_code = 'SEC' and
vendor_name = 'sqlserver' and ('${root_cause_id}' = '' OR root_cause_id = '${root_cause_id}')
```

Policy Violations by Type - *barchart*

```
SELECT COALESCE(i.name, gm.metric_name) AS violation_type, COUNT(*) AS count FROM monitoring.general_metric_metadata_results
gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id =
rc.issue_id WHERE (gm.metric_name LIKE 'SEC-SQL-AZ%%' OR gm.metric_name LIKE 'SEC-SQL-CFG%%' OR gm.metric_name LIKE
'SEC-SQL-AU%%' OR gm.metric_name LIKE 'SEC-SQL-NET%%' OR gm.metric_name LIKE 'SEC-SQL-ACC-010-RC08%%') AND
(gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY violation_type
ORDER BY count DESC LIMIT 15
```

5. Data Protection

Unmasked Columns - *stat*

```
SELECT COUNT(*) FROM monitoring.v_SEC_SQL_PRI_001_RC10
```

PII Columns - *stat*

```
SELECT COUNT(*) FROM monitoring.v_SEC_SQL_PRI_001_RC13
```

Encryption Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE (metric_name LIKE 'SEC-SQL-ENC%%' OR metric_name LIKE
'SEC-SQL-PRI%%') AND (metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Masking Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-DAT%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Unmasked Data by Server - *table*

```
SELECT * FROM monitoring.v_SEC_SQL_PRI_001_RC10
```

Data Protection - Unmasked Columns - *table*

```
SELECT * FROM monitoring.v_SEC_SQL_PRI_001_RC13 WHERE $__timeFilter(entry_date)
```

Sensitive Schema (PII Columns) - *table*

```
SELECT * from monitoring.v_SEC_SQL_PRI_001_RC12
```

Privileged Logins - *table*

```
SELECT DISTINCT root_cause_id, root_cause_name, domain_name, area_name FROM rootcause.v_rootcauses WHERE root_cause_name ~*
'(privilege)' ORDER BY root_cause_id;
```

Root cause details - ${root*cause*id} ${detail*server} ${detail*table} ${detail_column} - *table*

```
SELECT j.value AS result FROM jsonb_array_elements(
COALESCE(monitoring.get_root_cause_resultset(NULLIF('${root_cause_id}',''), $__timeFrom())::jsonb, '[]'::jsonb) ) AS j WHERE
(NULLIF('${detail_server}','') IS NULL OR j.value->>'server' = '${detail_server}') AND (NULLIF('${detail_table}','') IS NULL
OR j.value->>'table_name' = '${detail_table}') AND (NULLIF('${detail_column}','') IS NULL OR j.value->>'column_name' =
'${detail_column}') LIMIT 500
```

6. Vulnerability Assessment

Privileged Logins - *stat*

```
SELECT COUNT(*) FROM monitoring.privileged_logins
```

Weak Passwords - *stat*

```
SELECT COUNT(*) FROM monitoring.weak_passwords
```

Config Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-CFG%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Network Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-NET%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Total Vulnerabilities - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Vulnerabilities by Area - *barchart*

```
SELECT COALESCE(a.name, 'Unknown') AS area, COUNT(*) AS findings FROM monitoring.general_metric_metadata_results gm LEFT JOIN
rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id LEFT
JOIN rootcause.areas a ON a.code = i.area_code WHERE gm.metric_name LIKE 'SEC-SQL%%' AND
(gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY a.name ORDER BY
findings DESC LIMIT 15
```

By Severity - *piechart*

```
SELECT COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity, COUNT(*) AS count FROM
monitoring.general_metric_metadata_results gm WHERE gm.metric_name LIKE 'SEC-SQL%%' AND
(gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY severity ORDER BY
count DESC
```

Vulnerability Findings Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', gm.entry_date) AS time, COALESCE(a.name, 'Other') AS metric, COUNT(*) AS value FROM
monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT
JOIN rootcause.issues i ON i.issue_id = rc.issue_id LEFT JOIN rootcause.areas a ON a.code = i.area_code WHERE gm.metric_name
LIKE 'SEC-SQL%%' AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP
BY time, metric ORDER BY time
```

Privileged Logins - *table*

```
SELECT * FROM monitoring.privileged_logins ORDER BY entry_date DESC LIMIT 100
```

Server Hardening Findings - *table*

```
SELECT * FROM monitoring.server_hardening_unused_inactive_sql_server_logins ORDER BY entry_date DESC LIMIT 50
```

Vulnerability Root Cause Findings - *table*

```
SELECT gm.server, COALESCE(a.name, '') AS area, COALESCE(i.name, '') AS issue, gm.metric_name AS root_cause_id,
COALESCE(rc.name, gm.metric_name) AS root_cause, (gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity,
(gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE
'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON
rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id LEFT JOIN rootcause.areas a ON
a.code = i.area_code WHERE gm.metric_name LIKE 'SEC-SQL%%' AND gm.metric_metadata_vs_expected IS NOT NULL AND
(gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date
DESC LIMIT 100
```

Weak Passwords - *table*

```
SELECT * FROM monitoring.weak_passwords ORDER BY entry_date DESC LIMIT 50
```

7. Automation & Workflows

Servers - *stat*

```
SELECT COUNT(*) FROM metrics.servers WHERE is_active = true
```

Root Causes - *stat*

```
SELECT COUNT(DISTINCT root_cause_id) FROM rootcause.root_causes
```

Detection Rules - *stat*

```
SELECT COUNT(*) FROM rootcause.detection_paths WHERE is_active = true
```

Custom Metrics - *stat*

```
SELECT COUNT(*) FROM metrics.custom_metrics WHERE is_active = true
```

Active Transactions - *stat*

```
SELECT COUNT(*) FROM monitoring.active_transactions WHERE $__timeFilter(last_request_end_time)
```

Metric Results - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE $__timeFilter(entry_date)
```

Log Entries - *stat*

```
SELECT COUNT(*) FROM log.operation_log WHERE $__timeFilter(entry_date)
```

Metric Executions Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', entry_date) AS time, COUNT(*) AS executions FROM monitoring.general_metric_metadata_results WHERE
$__timeFilter(entry_date) GROUP BY time ORDER BY time
```

Root Causes - *table*

```
SELECT DISTINCT root_cause_id, root_cause_name, domain_name, area_name, issue_name, vendor_name FROM rootcause.v_rootcauses
ORDER BY root_cause_id LIMIT 100
```

Enable / Disable Metrics per Server - *table*

```
SELECT sr.row_id, s.server AS "IP Address", s.servername, r.routine_name AS metric_rule, CASE WHEN sr.is_active = true THEN
'Active' ELSE 'Inactive' END AS status FROM metrics.servers_routines sr JOIN metrics.servers s ON s.row_id = sr.server_id JOIN
metrics.routines r ON r.row_id = sr.routine_id ORDER BY s.server, r.routine_name
```

All Metrics (Latest Execution) - *table*

```
SELECT MAX(entry_date) AT TIME ZONE 'Asia/Jerusalem' AS last_run, metric_name FROM monitoring.general_metric_metadata_results
GROUP BY metric_name ORDER BY last_run DESC
```

Custom Metrics - *table*

```
SELECT row_id, metric_name, LEFT(query, 80) AS query_preview, description, is_active, db_vendor FROM metrics.custom_metrics
ORDER BY metric_name
```

Operation Log - *table*

```
SELECT entry_date AT TIME ZONE 'Asia/Jerusalem' AS time, routine_name, server_name, LEFT(message, 120) AS message FROM
log.operation_log WHERE $__timeFilter(entry_date) ORDER BY entry_date DESC LIMIT 50
```

Log Entries by Routine - *barchart*

```
SELECT routine_name, COUNT(*) AS entries FROM log.operation_log WHERE $__timeFilter(entry_date) GROUP BY routine_name ORDER BY
entries DESC LIMIT 15
```

Root cause details - ${root*cause*id} - *table*

```
SELECT j.value AS result FROM jsonb_array_elements(
COALESCE(monitoring.get_root_cause_resultset(NULLIF('${root_cause_id}',''), $__timeFrom())::jsonb, '[]'::jsonb) ) AS j LIMIT
500
```

8. Database restored

Database restored - *table*

```
select * from monitoring.v_database_restored limit 100--where $__timeFilter(backup_finish_date)
```

9. Connections per user

Connections per user - *table*

```
select cc.server , cc.login_name , cc.connection_count , "Average connection" from monitoring.connection_count cc join( select
server , login_name , AVG(connection_count::int) "Average connection" from monitoring.connection_count where login_name !=
'dbdome_mon_usr' group by server , login_name ) B on cc."server" = B.SERVER and cc.login_name = b.Login_name where
cc.login_name != 'dbdome_mon_usr' and $__timeFilter(entry_date)
```

Unkown TCP connections - *table*

```
select tcp.server , tcp.client_net_address , tcp.connect_time , at.login_name, at.query , at.start_time ,
at.last_request_end_time from ( select tcp.server , tcp.client_net_address , tcp.connect_time from monitoring.tcp_connections
tcp where client_net_address not in ( select client_net_address from monitoring.tcp_connections where connect_time < Now() -
interval '1 day' ) )tcp left outer join monitoring.active_transactions at on at.server = tcp.server and at.start_time between
tcp.connect_time and at.last_request_end_time where $__timeFilter(tcp.connect_time) and login_name !='dbdome_mon_usr'
```

Alerts

Total Alerts - *stat*

```
SELECT count(*) FROM ALERTS.ALERT_LOG l where l.entry_date >= $__timeFrom() at time zone current_setting('TimeZone') and
l.entry_date <= $__timeTo() at time zone current_setting('TimeZone') and root_cause_id like 'SEC-%'
```

Critical - *stat*

```
SELECT count(*) FROM ALERTS.ALERT_LOG l where l.entry_date >= $__timeFrom() at time zone current_setting('TimeZone') and
l.entry_date <= $__timeTo() at time zone current_setting('TimeZone') and l.risk_level = 'critical' and l.root_cause_ID like
'SEC-%'
```

High - *stat*

```
SELECT count(*) FROM ALERTS.ALERT_LOG l where l.entry_date >= $__timeFrom() at time zone current_setting('TimeZone') and
l.entry_date <= $__timeTo() at time zone current_setting('TimeZone') and l.risk_level = 'high' and l.root_cause_ID like
'SEC-%'
```

Medium - *stat*

```
SELECT count(*) FROM ALERTS.ALERT_LOG l where l.entry_date >= $__timeFrom() at time zone current_setting('TimeZone') and
l.entry_date <= $__timeTo() at time zone current_setting('TimeZone') and l.risk_level = 'medium' and l.root_cause_ID like
'SEC%'
```

Low alerts - *stat*

```
SELECT count(*) FROM ALERTS.ALERT_LOG l where l.entry_date >= $__timeFrom() at time zone current_setting('TimeZone') and
l.entry_date <= $__timeTo() at time zone current_setting('TimeZone') and l.risk_level = 'low' and l.root_cause_ID like 'SEC-%'
```

Servers - *stat*

```
select count(distinct(server_id)) from monitoring.general_metric_metadata_results
```

Alerts Sent by Mail - *table*

```
SELECT row_id , server, servername , risk_level , area, issue, root_cause, root_cause_name, recipients, occured_at occured_at
, sent_at sent_at , Blocked_at Blocked_at FROM ( SELECT ROW_NUMBER() OVER (PARTITION BY l.server, l.root_cause_id ORDER BY
L.ENTRY_DATE DESC )SEQ , l.row_id , l.server, s.servername , l.risk_level , COALESCE(rc.area_name, '') AS area,
COALESCE(rc.issue_name, '') AS issue, l.root_cause_id AS root_cause, rc.root_cause_name, mal.recipients, l.entry_date AS
occured_at, mal.entry_date sent_at , blc.entry_date Blocked_at FROM ALERTS.ALERT_LOG l join (select server_id , servername ,
server ,db_vendor from metrics.servers where is_active is true ) s on s.server = l.server LEFT JOIN alerts.mail_alert_log mal
ON mal.server = l.server AND mal.metric_name = l.root_cause_id and mal.entry_date = l.entry_date LEFT JOIN alerts.blocks blc
ON blc.server = l.server AND blc.metric_name = l.root_cause_id and blc.entry_date = l.entry_date JOIN rootcause.v_rootcauses
rc ON rc.root_cause_id = l.root_cause_id and (rc.vendor_name = s.db_vendor or rc.vendor_name ='sqlserver') AND (l.risk_level =
'${risk_level}' OR '${risk_level}' = 'All') ) WHERE SEQ=1 and occured_at >= $__timeFrom() at time zone
current_setting('TimeZone') and occured_at <= $__timeTo() at time zone current_setting('TimeZone')
```

Active Detection Findings - *table*

```
SELECT * FROM monitoring.get_alert_log_resultset_byid('${alert_id}') WHERE '${alert_id}' <> ''
```

Audit expired

Expired Audit Records - *stat*

```
SELECT COUNT(*) FROM monitoring.v_oracle_audit_expired
```

Servers Affected - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.v_oracle_audit_expired
```

Audit Root Causes - *stat*

```
SELECT COUNT(DISTINCT metric_name) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-AUD%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Audit Alerts Sent - *stat*

```
SELECT COUNT(*) FROM alerts.mail_alert_log WHERE metric_name LIKE 'SEC-SQL-AUD%%' AND $__timeFilter(entry_date)
```

Expired Audits by Server - *barchart*

```
SELECT server, COUNT(*) AS expired FROM monitoring.v_oracle_audit_expired GROUP BY server ORDER BY expired DESC
```

Top Audit Root Causes - *bargauge*

```
SELECT COALESCE(rc.name, gm.metric_name) AS root_cause, COUNT(*) AS count FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name WHERE gm.metric_name LIKE 'SEC-SQL-AUD%%' AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY root_cause ORDER BY count DESC LIMIT 10
```

Audit Expired Records - *table*

```
SELECT * FROM monitoring.v_oracle_audit_expired
```

Audit Root Cause Findings (SEC-SQL-AUD) - *table*

```
SELECT gm.server, COALESCE(i.name, '') AS issue, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS root_cause, (gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows, COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity, (gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE 'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id WHERE gm.metric_name LIKE 'SEC-SQL-AUD%%' AND gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date DESC LIMIT 50
```

Blocker Activity

Killed - *stat*

```
SELECT count(*) FROM alerts.blocker_log WHERE action='killed' AND $__timeFilter(entry_date)
```

Dry-run - *stat*

```
SELECT count(*) FROM alerts.blocker_log WHERE action='dry-run' AND $__timeFilter(entry_date)
```

Skipped - *stat*

```
SELECT count(*) FROM alerts.blocker_log WHERE action='skipped' AND $__timeFilter(entry_date)
```

Errors - *stat*

```
SELECT count(*) FROM alerts.blocker_log WHERE action='error' AND $__timeFilter(entry_date)
```

Dry-run mode - *stat*

```
SELECT value FROM config.global_params WHERE key='blocker_dry_run' ORDER BY row_id DESC LIMIT 1
```

Blocker log - *table*

```
SELECT entry_date AT TIME ZONE 'Asia/Jerusalem' AS entry_date, server, db_vendor, root_cause_id, session_id, action, detail FROM alerts.blocker_log WHERE $__timeFilter(entry_date) ORDER BY entry_date DESC LIMIT 500
```

Executed blocks (alerts.blocks) - *table*

```
SELECT entry_date AT TIME ZONE 'Asia/Jerusalem' AS entry_date, server, metric_name, subject, body FROM alerts.blocks ORDER BY entry_date DESC LIMIT 200
```

Blocking transactions

blocked transactions - *barchart*

```
select count(*) "number of blocked transactions" , server ||'-'||login_name from monitoring.v_blocking_transactions where login_name != 'infosecuser' group by server ||'-'||login_name
```

(untitled panel) - *table*

```
select distinct TO_CHAR(to_timestamp(start_time::bigint / 1000 ), 'YYYY-MM-DD') AS "start time" from
monitoring.v_blocking_transactions order by TO_CHAR(to_timestamp(start_time::bigint / 1000 ), 'YYYY-MM-DD')
```

Active users - *bargauge*

```
select sum("Number of connections") from ( select count(*) "Number of connections" from monitoring.tcp_connections where
connect_time between Now() - interval '1 minute' and Now() union all select count(*) from monitoring.v_oracle_transactions
where entry_date > Now() - interval '1 minute' )
```

SQL injection by patterns - *table*

```
select server , session_id , login_name , status , to_timestamp( start_time::bigint /1000) start_time, command , cpu_time ,
Total_elapsed_time::bigint / 1000000 Total_elapsed_time from monitoring.v_blocking_transactions
```

Recent alerts - SQL injections - *stat*

```
select (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and
query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR
1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
1=1 --''%', '%"" OR 1=1 --%']) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%']) as
"union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval
'1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%']) as "stacked_queries" , (select count(*)
from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%']) as "time_based_blind _injection" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%CONVERT%', '%CAST%']) as "error_based_injection" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%']) as "information_schema_probing" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
''=''%', '%'' OR ''x''=''x''%', '%admin''--%']) as "Authentication bypass patterns" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%']) as "encoding injection" , (select count(*) from monitoring.active_transactions
where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%']) as
"shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now()
- interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%']) as "ORDER BY_GROUP BY injection"
```

Connectivity reports - *barchart*

```
select count(*) , server ,client_net_address from monitoring.tcp_connections where connect_Time > Now() - interval '15
minutes' group by server ,client_net_address
```

Sensitivity reports - *barchart*

```
select count(*) , at.server from monitoring.active_transactions at join( select TABLE_NAME , column_name from
monitoring.v_schema where column_name ILIKE ANY
(array['%fname%','%lname%','%email%','%lastname%','%firstname%','%last_name%','%firstname%','%phone%','%ssn%','%dob%','%birth%','%credit%',
 ) c on at.query like '%'||c.column_name ||'%' group by at.server
```

SQL Injection analysis - *barchart*

```
select server , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute'
and query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR
1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
1=1 --''%', '%"" OR 1=1 --%']) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%']) as
"union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval
'1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%']) as "stacked_queries" , (select count(*)
from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%']) as "time_based_blind _injection" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%CONVERT%', '%CAST%']) as "error_based_injection" , (select count(*) from monitoring.active_transactions where
```

```
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.active_transactions
where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as
"shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now()
- interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection" from
monitoring.active_transactions group by server
```

Blocks in databases

Database Blocks - *barchart*

```
select count(*) , server from monitoring.v_mssql_blocking_sessiond group by server
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_mssql_blocking_sessiond
```

Active users - *bargauge*

```
select sum("Number of connections") from ( select count(*) "Number of connections" from monitoring.tcp_connections where
connect_time between Now() - interval '1 minute' and Now() union all select count(*) from monitoring.v_oracle_transactions
where entry_date > Now() - interval '1 minute' )
```

Database blocks - *table*

```
select * from monitoring.v_mssql_blocking_sessiond
```

Recent alerts - *stat*

```
select (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and
query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR
1=1%', '%AND 1=1%' , '%OR ''a''=''a%''])) as "Boolean-based injections" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as
"union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval
'1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*)
from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.active_transactions
where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as
"shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now()
- interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection" from
monitoring.active_transactions group by server
```

Connection number per user - *barchart*

```
select sum(connection_count::int) , server ||'-'||login_name from monitoring.connection_count group by server
||'-'||login_name
```

Active threats - *gauge*

```
select count(*) "suspicious queries" from monitoring.v_suspiscous
```

Configuration

Monitored Servers - *stat*

```
SELECT COUNT(*) AS "Servers" FROM metrics.servers WHERE is_active = true
```

Active Detection Rules - *stat*

```
SELECT COUNT(*) AS "Rules" FROM rootcause.detection_paths WHERE is_active = true
```

Alert Rules - *stat*

```
SELECT COUNT(*) AS "Alert Rules" FROM config.alerts_issue_root_causes
```

Sensitive Columns - *stat*

```
SELECT COUNT(*) AS "PII Columns" FROM monitoring.sensitive_schema
```

Scheduled Reports - *stat*

```
SELECT COUNT(*) AS "Reports" FROM config.reports WHERE is_active = true
```

Alerts Sent (7d) - *stat*

```
SELECT COUNT(*) AS "Alerts" FROM alerts.mail_alert_log WHERE entry_date >= NOW() - INTERVAL '7 days'
```

Monitored Servers - *table*

```
SELECT server, db_name AS database, port, db_vendor AS vendor, auth_type, is_active, server_id FROM metrics.servers ORDER BY server
```

Detection Rules by Vendor - *piechart*

```
SELECT vendor_slug AS vendor, COUNT(*) AS rules FROM rootcause.detection_paths WHERE is_active = true GROUP BY vendor_slug ORDER BY rules DESC
```

Detection Rules by Domain - *piechart*

```
SELECT d.name AS domain, COUNT(DISTINCT dp.id) AS rules FROM rootcause.detection_paths dp JOIN rootcause.root_causes rc ON rc.root_cause_id = dp.root_cause_id JOIN rootcause.issues i ON i.issue_id = rc.issue_id JOIN rootcause.domains d ON d.code = i.domain_code WHERE dp.is_active = true GROUP BY d.name ORDER BY rules DESC
```

Alerts Sent Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', entry_date) AS time, COUNT(*) AS alerts FROM alerts.mail_alert_log WHERE $__timeFilter(entry_date) GROUP BY time ORDER BY time
```

Alerts by Severity - *barchart*

```
SELECT CASE WHEN metric_name LIKE '%RC01' THEN 'Unknown Login' WHEN metric_name LIKE '%RC02' THEN 'Unexpected Program' WHEN metric_name LIKE '%RC03' THEN 'Unexpected DB' WHEN metric_name LIKE '%RC04' THEN 'Suspicious Patterns' WHEN metric_name LIKE '%RC05' THEN 'PII Access' WHEN metric_name LIKE '%RC06' THEN 'SQL Injection' WHEN metric_name LIKE '%RC07' THEN 'After Hours' WHEN metric_name LIKE '%RC08' THEN 'Privilege Escalation' WHEN metric_name LIKE '%RC09' THEN 'Data Exfiltration' WHEN metric_name LIKE '%RC10' THEN 'Multi-Host Login' WHEN metric_name LIKE '%RC11' THEN 'Schema Recon' WHEN metric_name LIKE '%RC12' THEN 'Dormant Account' WHEN metric_name LIKE '%RC13' THEN 'Mass Modification' ELSE metric_name END AS alert_type, COUNT(*) AS count FROM alerts.mail_alert_log WHERE $__timeFilter(entry_date) GROUP BY alert_type ORDER BY count DESC LIMIT 15
```

Configured Reports - *table*

```
SELECT r.report_name, r.is_active, COALESCE(j.occurance, 'not scheduled') AS schedule, j.occurs_at::text AS run_time, r.report_url AS type FROM config.reports r LEFT JOIN config.reports_jobs rj ON rj.report_id = r.row_id LEFT JOIN jobs.jobs j
```

```
ON j.row_id = rj.job_id ORDER BY r.report_name
```

Alert Thresholds - *table*

```
SELECT a.alert_name, t.level, CASE t.level WHEN 1 THEN 'Info' WHEN 2 THEN 'Warning' WHEN 3 THEN 'Critical' END AS severity, t.value_start, t.value_end FROM config.alerts_thresholds at JOIN config.alerts a ON a.row_id = at.alert_id JOIN config.thresholds t ON t.row_id = at.threshold_id ORDER BY t.level DESC, a.alert_name
```

Sensitive Schema - PII Columns - *table*

```
SELECT server, database_name, table_name, column_name, data_type, entry_date FROM monitoring.sensitive_schema ORDER BY server, table_name, column_name LIMIT 100
```

PII Columns by Category - *piechart*

```
SELECT CASE WHEN lower(column_name) ~ '(email|e_mail)' THEN 'Email' WHEN lower(column_name) ~ '(phone|mobile|cell)' THEN 'Phone' WHEN lower(column_name) ~ '(first_name|last_name|full_name|surname)' THEN 'Name' WHEN lower(column_name) ~ '(address|street|city|zip)' THEN 'Address' WHEN lower(column_name) ~ '(ssn|national_id|passport|id_card)' THEN 'ID/SSN' WHEN lower(column_name) ~ '(credit_card|card_num|iban|bank)' THEN 'Financial' WHEN lower(column_name) ~ '(salary|income|wage)' THEN 'Salary' WHEN lower(column_name) ~ '(password|pwd|secret)' THEN 'Credential' WHEN lower(column_name) ~ '(birth|dob)' THEN 'DOB' ELSE 'Other' END AS category, COUNT(*) AS count FROM monitoring.sensitive_schema GROUP BY category ORDER BY count DESC
```

Webhook Alert Channels - *table*

```
SELECT metric_type, CASE WHEN send_mail_alert THEN 'ON' ELSE 'OFF' END AS mail, CASE WHEN send_siem_alert THEN 'ON' ELSE 'OFF' END AS siem, CASE WHEN send_diagnosis_evidence THEN 'ON' ELSE 'OFF' END AS diagnosis FROM config.webook_alerts ORDER BY metric_type
```

Global Parameters - *table*

```
SELECT key, CASE WHEN key ILIKE '%password%' OR key ILIKE '%api_key%' OR key ILIKE '%secret%' THEN '********' ELSE value END AS value FROM config.global_params ORDER BY key
```

Recent Operation Log - *table*

```
SELECT entry_date AS time, routine_name, server_name, LEFT(message, 150) AS message FROM log.operation_log WHERE $__timeFilter(entry_date) ORDER BY entry_date DESC LIMIT 50
```

Detection Results Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', entry_date) AS time, SUM(CASE WHEN (metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' THEN 1 ELSE 0 END) AS matched, SUM(CASE WHEN (metric_metadata_vs_expected::jsonb->>'matched')::text = 'false' THEN 1 ELSE 0 END) AS not_matched FROM monitoring.general_metric_metadata_results WHERE metric_metadata_vs_expected IS NOT NULL AND $__timeFilter(entry_date) GROUP BY time ORDER BY time
```

Top Root Causes by Findings - *barchart*

```
SELECT metric_name AS root_cause, COUNT(*) AS findings FROM monitoring.general_metric_metadata_results WHERE metric_metadata_vs_expected IS NOT NULL AND (metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date) GROUP BY metric_name ORDER BY findings DESC LIMIT 15
```

Rootcause Taxonomy Overview - *table*

```
SELECT d.name AS domain, a.name AS area, i.issue_id, i.name AS issue, COUNT(DISTINCT rc.root_cause_id) AS root_causes, COUNT(DISTINCT dp.id) AS detection_paths, COUNT(DISTINCT rp.id) AS resolution_paths FROM rootcause.domains d JOIN rootcause.issues i ON i.domain_code = d.code JOIN rootcause.areas a ON a.code = i.area_code JOIN rootcause.root_causes rc ON rc.issue_id = i.issue_id LEFT JOIN rootcause.detection_paths dp ON dp.root_cause_id = rc.root_cause_id AND dp.is_active = true LEFT JOIN rootcause.resolution_paths rp ON rp.root_cause_id = rc.root_cause_id AND rp.is_active = true WHERE d.is_enabled = true GROUP BY d.name, a.name, i.issue_id, i.name ORDER BY d.name, a.name, i.issue_id LIMIT 100
```

Mail Configuration - *table*

```
SELECT row_id, mail_sender, smtp_server, smtp_port, CASE WHEN smtp_user IS NOT NULL THEN 'Configured' ELSE 'None' END AS auth, tls FROM config.mail_config ORDER BY row_id
```

Mail Groups & Recipients - *table*

```
SELECT row_id, mail_config_id, group_name, recipients, is_active FROM config.mail_groups ORDER BY group_name
```

Risk Levels - *table*

```
SELECT row_id, trim(risk_level) AS risk_level, is_active FROM rootcause.risk_level ORDER BY row_id
```

Print (filtered view) - *table*

```
SELECT ' Print this filtered view (opens clean tab -> Ctrl+P / Save as PDF)' AS print
```

Connectivity

Active Connections - *stat*

```
SELECT COUNT(*) FROM monitoring.tcp_connections WHERE connect_time > NOW() - INTERVAL '1 hour'
```

Unique Clients - *stat*

```
SELECT COUNT(DISTINCT client_net_address) FROM monitoring.tcp_connections WHERE connect_time > NOW() - INTERVAL '1 hour'
```

Servers - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.tcp_connections WHERE connect_time > NOW() - INTERVAL '1 hour'
```

Network Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-NET%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Connection Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'PERF-SQL-CN%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Connections by Server & Client - *barchart*

```
SELECT server || ' <- ' || client_net_address AS connection, COUNT(*) AS count FROM monitoring.tcp_connections WHERE
connect_time > NOW() - INTERVAL '1 hour' GROUP BY server, client_net_address ORDER BY count DESC LIMIT 15
```

By Protocol - *piechart*

```
SELECT COALESCE(protocol_type, 'unknown') AS protocol, COUNT(*) AS count FROM monitoring.tcp_connections WHERE connect_time >
NOW() - INTERVAL '1 hour' GROUP BY protocol_type
```

By Auth Scheme - *piechart*

```
SELECT COALESCE(auth_scheme, 'unknown') AS auth, COUNT(*) AS count FROM monitoring.tcp_connections WHERE connect_time > NOW()
- INTERVAL '1 hour' GROUP BY auth_scheme
```

TCP Connections - *table*

```
SELECT server, connect_time, net_transport, protocol_type, encrypt_option, auth_scheme, client_net_address, num_reads,
num_writes, last_read, last_write FROM monitoring.tcp_connections WHERE connect_time > NOW() - INTERVAL '1 hour' ORDER BY
connect_time DESC LIMIT 200
```

Network & Connection Root Cause Findings - *table*

```
SELECT gm.server, COALESCE(i.name, '') AS issue, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS
root_cause, (gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity,
(gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE
'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON
rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id WHERE (gm.metric_name LIKE
'SEC-SQL-NET%%' OR gm.metric_name LIKE 'PERF-SQL-CN%%' OR gm.metric_name LIKE 'SEC-SQL-ACC-010-RC10%%') AND
gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND
$__timeFilter(gm.entry_date) ORDER BY gm.entry_date DESC LIMIT 50
```

Data Activity (Transactions/hour) - *barchart*

```
SELECT server, COUNT(*) AS transactions FROM monitoring.active_transactions WHERE last_request_end_time > NOW() - INTERVAL '1
hour' GROUP BY server ORDER BY transactions DESC
```

Connections per Client - *barchart*

```
SELECT server || ' <- ' || client_net_address AS client, COUNT(*) AS connections FROM monitoring.tcp_connections WHERE
connect_time > NOW() - INTERVAL '1 hour' GROUP BY server, client_net_address ORDER BY connections DESC LIMIT 10
```

Credentials stored in tables

Credentials Found - *stat*

```
SELECT COUNT(*) FROM monitoring.v_mssql_credentials_stored_in_tables
```

Servers Affected - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.v_mssql_credentials_stored_in_tables
```

Tables with Credentials - *stat*

```
SELECT COUNT(DISTINCT table_name) FROM monitoring.v_mssql_credentials_stored_in_tables
```

Root Cause Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-PRI%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Credentials by Server & Table - *barchart*

```
SELECT server || ' - ' || table_name AS location, COUNT(*) AS count FROM monitoring.v_mssql_credentials_stored_in_tables GROUP
BY server, table_name ORDER BY count DESC LIMIT 15
```

By Server - *piechart*

```
SELECT server, COUNT(*) AS count FROM monitoring.v_mssql_credentials_stored_in_tables GROUP BY server ORDER BY count DESC
```

Credentials Stored in Database Tables - *table*

```
SELECT * FROM monitoring.v_mssql_credentials_stored_in_tables
```

PII/Credential Root Cause Findings - *table*

```
SELECT gm.server, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS root_cause,
(gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity,
(gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE
'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON
rc.root_cause_id = gm.metric_name WHERE (gm.metric_name LIKE 'SEC-SQL-PRI%%' OR gm.metric_name LIKE 'SEC-SQL-ENC%%') AND
gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND
$__timeFilter(gm.entry_date) ORDER BY gm.entry_date DESC LIMIT 50
```

Custom metrics

Total Metrics - *stat*

```
SELECT COUNT(*) FROM metrics.custom_metrics
```

Active Metrics - *stat*

```
SELECT COUNT(*) FROM metrics.custom_metrics WHERE is_active = true
```

Inactive Metrics - *stat*

```
SELECT COUNT(*) FROM metrics.custom_metrics WHERE is_active = false
```

Metric Results (period) - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE $__timeFilter(entry_date)
```

Matched Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_metadata_vs_expected IS NOT NULL AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Metrics by Vendor - *piechart*

```
SELECT COALESCE(db_vendor, 'unknown') AS vendor, COUNT(*) AS count FROM metrics.custom_metrics GROUP BY db_vendor ORDER BY
count DESC
```

By Type - *piechart*

```
SELECT COALESCE(metrics_type, 'standard') AS type, COUNT(*) AS count FROM metrics.custom_metrics GROUP BY metrics_type ORDER
BY count DESC
```

Top Executed Metrics - *barchart*

```
SELECT metric_name, COUNT(*) AS executions FROM monitoring.general_metric_metadata_results WHERE $__timeFilter(entry_date)
GROUP BY metric_name ORDER BY executions DESC LIMIT 15
```

Custom Metrics Configuration - *table*

```
SELECT row_id, metric_name, LEFT(query, 100) AS query_preview, description, is_active, db_vendor, metrics_type, entry_date AT
TIME ZONE 'Asia/Jerusalem' AS created FROM metrics.custom_metrics ORDER BY metric_name
```

Metric Results with Comparison - *table*

```
SELECT server, metric_name, (metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
(metric_metadata_vs_expected::jsonb->>'matched')::text AS matched, COALESCE(metric_metadata_vs_expected::jsonb->>'severity',
'') AS severity, metric_metadata_vs_expected::jsonb->>'condition' AS condition, entry_date AT TIME ZONE 'Asia/Jerusalem' AS
executed_at FROM monitoring.general_metric_metadata_results WHERE metric_metadata_vs_expected IS NOT NULL AND
$__timeFilter(entry_date) ORDER BY entry_date DESC LIMIT 100
```

Metric Executions Over Time - *timeseries*

```
SELECT DATE_TRUNC('hour', entry_date) AS time, COUNT(*) AS executions FROM monitoring.general_metric_metadata_results WHERE
$__timeFilter(entry_date) GROUP BY time ORDER BY time
```

Database connections per user

Total Connections - *stat*

```
SELECT SUM(connection_count::int) FROM monitoring.connection_count
```

Unique Users - *stat*

```
SELECT COUNT(DISTINCT login_name) FROM monitoring.connection_count
```

Servers - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.connection_count
```

Multi-Host Alerts - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE '%%ACC-010-RC10%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Connection Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'PERF-SQL-CN%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Connections per User - *barchart*

```
SELECT server || ' - ' || login_name AS user_server, SUM(connection_count::int) AS connections FROM
monitoring.connection_count GROUP BY server, login_name ORDER BY connections DESC LIMIT 20
```

By Server - *piechart*

```
SELECT server, SUM(connection_count::int) AS connections FROM monitoring.connection_count GROUP BY server ORDER BY connections
DESC
```

Connections per User (Detail) - *table*

```
SELECT * FROM monitoring.connection_count ORDER BY connection_count::int DESC
```

Connection Root Cause Findings - *table*

```
SELECT gm.server, COALESCE(i.name, '') AS issue, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS
root_cause, (gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows,
COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity,
(gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE
'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON
rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id WHERE (gm.metric_name LIKE
'PERF-SQL-CN%%' OR gm.metric_name LIKE '%%ACC-010-RC10%%') AND gm.metric_metadata_vs_expected IS NOT NULL AND
(gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date
DESC LIMIT 50
```

Database restored

Database restored - *gauge*

```
select count(*) from monitoring.v_database_restored
```

(untitled panel) - *table*

```
select server from monitoring.v_database_restored
```

Active users - *bargauge*

```
select sum("Number of connections") from ( select count(*) "Number of connections" from monitoring.tcp_connections where
connect_time between Now() - interval '1 minute' and Now() union all select count(*) from monitoring.v_oracle_transactions
where entry_date > Now() - interval '1 minute' )
```

Recent alerts - *stat*

```
select (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and
query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR
1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
1=1 --''%', '%"" OR 1=1 --%']) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%']) as
"union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval
'1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%']) as "stacked_queries" , (select count(*)
from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%']) as "time_based_blind _injection" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%CONVERT%', '%CAST%']) as "error_based_injection" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%']) as "information_schema_probing" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
''=''%', '%'' OR ''x''=''x''%', '%admin''--%']) as "Authentication bypass patterns" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%']) as "encoding injection" , (select count(*) from monitoring.active_transactions
where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%']) as
"shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now()
- interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%']) as "ORDER BY_GROUP BY injection" from
monitoring.active_transactions group by server
```

Database restored - *table*

```
select * from monitoring.v_database_restored
```

Connection number per user - *barchart*

```
select sum(connection_count::int) , server ||'-'||login_name from monitoring.connection_count group by server
||'-'||login_name
```

Active threats - *gauge*

```
select count(*) "suspicious queries" from monitoring.v_suspiscous
```

Email configuration

Email configuration - *table*

```
select mc.row_id , smtp_server , smtp_password , smtp_port , tls , mg.recipients from config.mail_config mc join
config.mail_groups mg on mg.mail_config_id = mc.row_id
```

Mail Groups & Recipients - *table*

```
SELECT row_id, mail_config_id, group_name, recipients, is_active FROM config.mail_groups ORDER BY group_name
```

Enabled sysadmin

Enabled sysadmin - *barchart*

```
select count(*) ,server from monitoring.v_enabled_sysadmin group by server
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_enabled_sysadmin
```

Active users - *bargauge*

```
select sum("Number of connections") from ( select count(*) "Number of connections" from monitoring.tcp_connections where connect_time between Now() - interval '1 minute' and Now() union all select count(*) from monitoring.v_oracle_transactions where entry_date > Now() - interval '1 minute' )
```

Recent alerts - *stat*

```
select (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR 1=1%', '%AND 1=1%' , '%OR ''a''=''a%''])) as "Boolean-based injections" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%', '%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection" from monitoring.active_transactions group by server
```

Enabled sysadmin - *table*

```
select * from monitoring.v_enabled_sysadmin where $__timeFilter(entry_date)
```

Connection number per user - *barchart*

```
select sum(connection_count::int) , server ||'-'||login_name from monitoring.connection_count group by server ||'-'||login_name
```

Active threats - *gauge*

```
select count(*) "suspicious queries" from monitoring.v_suspiscous
```

GRC Access Review

Open Review Instances - *stat*

```
SELECT COUNT(*) FROM log.access_review_instances WHERE status='OPEN'
```

Overdue Reviews - *stat*

```
SELECT COUNT(*) FROM log.access_review_instances WHERE status='OPEN' AND period_end < NOW()
```

Completed (90d) - *stat*

```
SELECT COUNT(*) FROM log.access_review_instances WHERE status='COMPLETE' AND completed_at >= NOW()-INTERVAL '90 days'
```

Active Cycles - *stat*

```
SELECT COUNT(*) FROM config.access_review_cycles WHERE is_active=TRUE
```

Reviews Opened vs Completed per Month - *timeseries*

```
SELECT date_trunc('month', created_at) AS "time", SUM(CASE WHEN status IN ('OPEN','COMPLETE','CANCELLED') THEN 1 ELSE 0 END)
AS "Opened", SUM(CASE WHEN status='COMPLETE' THEN 1 ELSE 0 END) AS "Completed" FROM log.access_review_instances WHERE
$__timeFilter(created_at) GROUP BY 1 ORDER BY 1
```

Instances by Status - *piechart*

```
SELECT status, COUNT(*) AS count FROM log.access_review_instances GROUP BY status
```

Open Reviews by Cycle - *barchart*

```
SELECT cycle_name, COUNT(*) AS open_instances FROM log.access_review_instances WHERE status='OPEN' GROUP BY cycle_name ORDER
BY open_instances DESC LIMIT 10
```

Avg Days to Complete by Cycle - *barchart*

```
SELECT cycle_name, ROUND(AVG(EXTRACT(EPOCH FROM (completed_at - created_at))/86400)::numeric, 1) AS avg_days FROM
log.access_review_instances WHERE status='COMPLETE' GROUP BY cycle_name ORDER BY avg_days DESC LIMIT 10
```

Overdue Review Instances - Action Required - *table*

```
sELECT i.instance_id, i.cycle_name, i.period_start, i.period_end, NOW()::date - i.period_end::date AS days_overdue,
COALESCE(NULLIF(array_to_string(c.scope_regulations, ', '), ''), '-') AS regulation, i.created_at FROM
log.access_review_instances i JOIN config.access_review_cycles c ON c.cycle_id = i.cycle_id WHERE i.status='OPEN' AND
i.period_end < NOW() ORDER BY i.period_end ASC LIMIT 200
```

All Open Review Instances - *table*

```
SELECT i.instance_id, i.cycle_name, i.status, i.period_start, i.period_end, CASE WHEN i.period_end < NOW() THEN 'OVERDUE' ELSE
'ON-TIME' END AS deadline, COALESCE(NULLIF(array_to_string(c.scope_regulations, ', '), ''), '') AS regulation, i.created_at
FROM log.access_review_instances i JOIN config.access_review_cycles c ON c.cycle_id = i.cycle_id WHERE i.status='OPEN' ORDER
BY i.period_end ASC LIMIT 500
```

Review Cycle Configuration - *table*

```
SELECT cycle_id, cycle_name, frequency, next_cycle_due_at AS next_run, COALESCE(NULLIF(array_to_string(scope_regulations, ',
'), ''), '-') AS regulation, CASE WHEN is_active THEN 'Active' ELSE 'Inactive' END AS status FROM config.access_review_cycles
ORDER BY cycle_name
```

GRC Audit Log

ALERTED Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE action_taken='ALERTED' AND $__timeFilter(event_time)
```

BLOCKED Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE action_taken='BLOCKED' AND $__timeFilter(event_time)
```

MASKED Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE action_taken='MASKED' AND $__timeFilter(event_time)
```

Events per Hour by Action - *timeseries*

```
SELECT date_trunc('hour', event_time) AS "time", SUM(CASE WHEN action_taken='BLOCKED' THEN 1 ELSE 0 END) AS "BLOCKED",
SUM(CASE WHEN action_taken='ALERTED' THEN 1 ELSE 0 END) AS "ALERTED", SUM(CASE WHEN action_taken='MASKED' THEN 1 ELSE 0 END)
AS "MASKED", SUM(CASE WHEN action_taken='ALLOWED' THEN 1 ELSE 0 END) AS "ALLOWED" FROM log.firewall_audit_log WHERE
$__timeFilter(event_time) GROUP BY 1 ORDER BY 1
```

Top Source IPs (BLOCKED) - *barchart*

```
SELECT COALESCE(client_ip,'unknown') AS ip, COUNT(*) AS events FROM log.firewall_audit_log WHERE action_taken='BLOCKED' AND $__timeFilter(event_time) GROUP BY client_ip ORDER BY events DESC LIMIT 10
```

Top Users (BLOCKED) - *barchart*

```
SELECT COALESCE(db_user,'unknown') AS db_user, COUNT(*) AS events FROM log.firewall_audit_log WHERE action_taken='BLOCKED' AND $__timeFilter(event_time) GROUP BY db_user ORDER BY events DESC LIMIT 10
```

Top Servers (all actions) - *barchart*

```
SELECT COALESCE(server_name,'unknown') AS server, COUNT(*) AS events FROM log.firewall_audit_log WHERE $__timeFilter(event_time) GROUP BY server_name ORDER BY events DESC LIMIT 10
```

Audit Events - *table*

```
SELECT a.event_time, a.server_name, a.db_user, a.client_ip, a.db_name, a.action_taken, a.risk_score, a.regulation, p.policy_name, left(a.sql_statement, 120) AS sql_statement FROM log.firewall_audit_log a LEFT JOIN config.firewall_policies p ON p.policy_id = a.matched_policy WHERE $__timeFilter(a.event_time) ORDER BY a.event_time DESC LIMIT 200
```

GRC Compliance

PCI-DSS Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE regulation='PCI-DSS' AND $__timeFilter(event_time)
```

HIPAA Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE regulation='HIPAA' AND $__timeFilter(event_time)
```

GDPR Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE regulation='GDPR' AND $__timeFilter(event_time)
```

SOC2 Events - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE regulation='SOC2' AND $__timeFilter(event_time)
```

Events by Regulation (hourly) - *timeseries*

```
SELECT date_trunc('hour', event_time) AS "time", SUM(CASE WHEN regulation='PCI-DSS' THEN 1 ELSE 0 END) AS "PCI-DSS", SUM(CASE WHEN regulation='HIPAA' THEN 1 ELSE 0 END) AS "HIPAA", SUM(CASE WHEN regulation='GDPR' THEN 1 ELSE 0 END) AS "GDPR", SUM(CASE WHEN regulation='SOC2' THEN 1 ELSE 0 END) AS "SOC2" FROM log.firewall_audit_log WHERE $__timeFilter(event_time) AND regulation IS NOT NULL GROUP BY 1 ORDER BY 1
```

Actions by Regulation - *barchart*

```
SELECT regulation, COUNT(*) AS events FROM log.firewall_audit_log WHERE $__timeFilter(event_time) AND regulation IS NOT NULL GROUP BY regulation ORDER BY events DESC
```

Regulation Breach Events (BLOCKED / ALERTED) - *table*

```
SELECT a.event_time, a.server_name, a.db_user, a.client_ip, a.action_taken, a.regulation, a.risk_score, p.policy_name FROM log.firewall_audit_log a LEFT JOIN config.firewall_policies p ON p.policy_id = a.matched_policy WHERE a.action_taken IN ('BLOCKED','ALERTED') AND a.regulation IS NOT NULL AND $__timeFilter(a.event_time) ORDER BY a.event_time DESC LIMIT 100
```

Masking Coverage by Regulation - *table*

```
SELECT regulation, COUNT(*) AS total_rules, SUM(CASE WHEN is_active THEN 1 ELSE 0 END) AS active_rules, array_to_string(array_agg(DISTINCT pii_type ORDER BY pii_type), ', ') AS pii_types FROM config.masking_rules WHERE regulation IS NOT NULL GROUP BY regulation ORDER BY regulation
```

Daily Audit Summary by Action - *table*

```
SELECT DATE(event_time) AS report_date, action_taken, COUNT(*) AS event_count, COUNT(DISTINCT db_user) AS unique_users,
COUNT(DISTINCT server_name) AS servers, MAX(risk_score) AS max_risk FROM log.firewall_audit_log WHERE
$__timeFilter(event_time) GROUP BY 1, 2 ORDER BY 1 DESC, event_count DESC
```

GRC Control Attestation Workflow

Pending Attestations - *stat*

```
SELECT COUNT(*) FROM log.attestations WHERE attestation_status='PENDING'
```

Exception Attestations - *stat*

```
SELECT COUNT(*) FROM log.attestations WHERE attestation_status='EXCEPTION'
```

Open Periods - *stat*

```
SELECT COUNT(*) FROM config.attestation_periods WHERE status='OPEN'
```

Completion Rate (Latest Period) - *stat*

```
SELECT ROUND(100.0*SUM(CASE WHEN attestation_status!='PENDING' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)::numeric,1) FROM
log.attestations WHERE period_id=(SELECT MAX(period_id) FROM log.attestations)
```

Attestations Completed per Month - *timeseries*

```
SELECT date_trunc('month', attested_at) AS "time", COUNT(*) AS attested FROM log.attestations WHERE
attestation_status!='PENDING' AND $__timeFilter(attested_at) GROUP BY 1 ORDER BY 1
```

Attestation Status Distribution - *piechart*

```
SELECT attestation_status AS status, COUNT(*) AS count FROM log.attestations GROUP BY attestation_status
```

Pending by Regulation - *barchart*

```
SELECT regulation, COUNT(*) AS pending FROM log.attestations WHERE attestation_status='PENDING' GROUP BY regulation ORDER BY
pending DESC
```

Completion Rate - SOC2 - *gauge*

```
SELECT ROUND(100.0*SUM(CASE WHEN attestation_status!='PENDING' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)::numeric,1) FROM
log.attestations WHERE regulation='SOC2'
```

Completion Rate - SOX - *gauge*

```
SELECT ROUND(100.0*SUM(CASE WHEN attestation_status!='PENDING' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)::numeric,1) FROM
log.attestations WHERE regulation='SOX'
```

Completion Rate - PCI-DSS - *gauge*

```
SELECT ROUND(100.0*SUM(CASE WHEN attestation_status!='PENDING' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)::numeric,1) FROM
log.attestations WHERE regulation='PCI-DSS'
```

Completion Rate - GDPR - *gauge*

```
SELECT ROUND(100.0*SUM(CASE WHEN attestation_status!='PENDING' THEN 1 ELSE 0 END)/NULLIF(COUNT(*),0)::numeric,1) FROM
log.attestations WHERE regulation='GDPR'
```

Pending Attestations - Action Required - *table*

```
SELECT a.attestation_id, p.period_name, p.due_date, a.control_name, a.regulation, CASE WHEN p.due_date<CURRENT_DATE THEN
'OVERDUE' ELSE 'ON-TIME' END AS deadline, a.created_at FROM log.attestations a JOIN config.attestation_periods p ON
p.period_id=a.period_id WHERE a.attestation_status='PENDING' ORDER BY p.due_date ASC, a.regulation, a.control_name LIMIT 500
```

Attestation Exceptions - *table*

```
SELECT a.attestation_id, p.period_name, a.control_name, a.regulation, a.attested_by, a.attested_at, a.exception_description
FROM log.attestations a JOIN config.attestation_periods p ON p.period_id=a.period_id WHERE a.attestation_status='EXCEPTION'
ORDER BY a.attested_at DESC LIMIT 200
```

GRC Cross-Border Transfer Tracking

Transfers Detected (30d) - *stat*

```
SELECT COUNT(*) FROM log.cross_border_transfers WHERE detected_at>=NOW()-INTERVAL '30 days'
```

Without Legal Basis (30d) - *stat*

```
SELECT COUNT(*) FROM log.cross_border_transfers WHERE has_legal_basis=FALSE AND detected_at>=NOW()-INTERVAL '30 days'
```

Critical Risk Transfers - *stat*

```
SELECT COUNT(*) FROM log.cross_border_transfers WHERE risk_level='CRITICAL' AND detected_at>=NOW()-INTERVAL '30 days'
```

Server Regions Configured - *stat*

```
SELECT COUNT(*) FROM config.data_regions
```

Cross-Border Transfers per Day - *timeseries*

```
SELECT date_trunc('day', detected_at) AS "time", SUM(CASE WHEN has_legal_basis THEN 1 ELSE 0 END) AS "With Legal Basis",
SUM(CASE WHEN NOT has_legal_basis THEN 1 ELSE 0 END) AS "No Legal Basis" FROM log.cross_border_transfers WHERE
$__timeFilter(detected_at) GROUP BY 1 ORDER BY 1
```

Transfer Mechanisms Used - *piechart*

```
SELECT COALESCE(transfer_mechanism,'NONE') AS mechanism, COUNT(*) AS transfers FROM log.cross_border_transfers WHERE
detected_at>=NOW()-INTERVAL '30 days' GROUP BY mechanism
```

Transfers by Destination Country (30d) - *barchart*

```
SELECT COALESCE(destination_country,'Unknown') AS country, COUNT(*) AS transfers FROM log.cross_border_transfers WHERE
detected_at>=NOW()-INTERVAL '30 days' GROUP BY country ORDER BY transfers DESC LIMIT 10
```

Transfer Agreements - *table*

```
SELECT source_server, destination_server, agreement_type, valid_from, valid_until FROM config.transfer_agreements ORDER BY
source_server, destination_server
```

Transfers Without Legal Basis - Action Required - *table*

```
SELECT transfer_id, source_server, destination_server, source_country, destination_country, transfer_mechanism, db_user,
risk_level, detected_at FROM log.cross_border_transfers WHERE has_legal_basis=FALSE ORDER BY detected_at DESC LIMIT 200
```

Server Region Registry - *table*

```
SELECT server_name, country_code, country_name, data_residency_zone, CASE WHEN is_adequate THEN 'YES' ELSE 'NO' END AS
adequate FROM config.data_regions ORDER BY data_residency_zone, server_name
```

GRC Immutable Audit Hash Chain

Total Rows Hashed - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE regulation is not null
```

Rows Pending Hash - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE regulation IS NULL
```

Last Verification Result - *stat*

```
SELECT CASE WHEN is_valid THEN 1 ELSE 0 END FROM log.hash_chain_checkpoints ORDER BY verified_at DESC LIMIT 1
```

Verifications Run - *stat*

```
SELECT COUNT(*) FROM log.hash_chain_checkpoints
```

Recent Audit Log - Hash Status - *table*

```
SELECT audit_id, event_time, server_name, db_user, action_taken, LEFT(row_hash,24)||'...' AS row_hash, CASE WHEN row_hash IS NOT NULL THEN 'HASHED' ELSE 'PENDING' END AS status FROM log.firewall_audit_log ORDER BY audit_id DESC LIMIT 200
```

Rows Hashed per Hour - *timeseries*

```
SELECT date_trunc('hour', event_time) AS "time", COUNT(*) AS rows_hashed FROM log.firewall_audit_log WHERE row_hash IS NOT NULL AND $__timeFilter(event_time) GROUP BY 1 ORDER BY 1
```

Verification Results (Last 30) - *barchart*

```
SELECT CASE WHEN is_valid THEN 'VALID' ELSE 'INVALID' END AS result, COUNT(*) AS count FROM log.hash_chain_checkpoints group by is_valid , verified_at ORDER BY verified_at DESC LIMIT 30
```

Hash Chain Verification Checkpoints - *table*

```
SELECT checkpoint_id, verified_at, rows_hashed, last_hashed_id, LEFT(last_hash,24)||'...' AS chain_tip, CASE WHEN is_valid THEN 'VALID' ELSE 'INVALID' END AS result, broken_at_id, error_detail FROM log.hash_chain_checkpoints ORDER BY verified_at DESC LIMIT 100
```

GRC Incident Lifecycle Management

Active Incidents - *stat*

```
SELECT COUNT(*) FROM log.incidents WHERE state NOT IN ('CLOSED')
```

Critical Active - *stat*

```
SELECT COUNT(*) FROM log.incidents WHERE severity='CRITICAL' AND state NOT IN ('CLOSED')
```

GDPR 72h Overdue - *stat*

```
SELECT COUNT(*) FROM log.incidents WHERE involves_personal_data=TRUE AND gdpr_notification_due<NOW() AND gdpr_notified_at IS NULL AND state!='CLOSED'
```

CAPA Actions Open - *stat*

```
SELECT COUNT(*) FROM log.incident_capa WHERE status='OPEN'
```

Incidents per Week by Severity - *timeseries*

```
SELECT date_trunc('week', detected_at) AS "time", SUM(CASE WHEN severity='CRITICAL' THEN 1 ELSE 0 END) AS "CRITICAL", SUM(CASE WHEN severity='HIGH' THEN 1 ELSE 0 END) AS "HIGH", SUM(CASE WHEN severity='MEDIUM' THEN 1 ELSE 0 END) AS "MEDIUM", SUM(CASE
```

```
WHEN severity='LOW' THEN 1 ELSE 0 END) AS "LOW" FROM log.incidents WHERE $__timeFilter(detected_at) GROUP BY 1 ORDER BY 1
```

Incidents by State - *piechart*

```
SELECT state, COUNT(*) AS incidents FROM log.incidents GROUP BY state
```

Incidents by Category (90d) - *barchart*

```
SELECT COALESCE(category,'other') AS category, COUNT(*) AS incidents FROM log.incidents WHERE detected_at>=NOW()-INTERVAL '90 days' GROUP BY category ORDER BY incidents DESC
```

Avg Resolution Time by Severity (h) - *barchart*

```
SELECT severity, ROUND(AVG(EXTRACT(EPOCH FROM (closed_at-detected_at))/3600)::numeric,1) AS avg_hours FROM log.incidents WHERE state='CLOSED' AND closed_at IS NOT NULL GROUP BY severity ORDER BY avg_hours DESC
```

GDPR Breach Notifications - *table*

```
SELECT incident_id, title, severity, detected_at, gdpr_notification_due, CASE WHEN gdpr_notified_at IS NOT NULL THEN 'NOTIFIED' WHEN gdpr_notification_due<NOW() THEN 'OVERDUE' ELSE 'PENDING' END AS gdpr_status, gdpr_notified_at FROM log.incidents WHERE involves_personal_data=TRUE ORDER BY gdpr_notification_due ASC NULLS LAST LIMIT 100
```

Active Incidents - *table*

```
SELECT incident_id, title, severity, state, category, detected_at, assigned_to, CASE WHEN involves_personal_data THEN 'YES' ELSE '' END AS gdpr FROM log.incidents WHERE state NOT IN ('CLOSED') ORDER BY CASE severity WHEN 'CRITICAL' THEN 1 WHEN 'HIGH' THEN 2 WHEN 'MEDIUM' THEN 3 ELSE 4 END, detected_at DESC LIMIT 200
```

Open CAPA Actions - *table*

```
SELECT c.capa_id, c.incident_id, i.title AS incident_title, c.capa_type, c.description, c.assigned_to, c.due_date, c.status, CASE WHEN c.due_date<CURRENT_DATE THEN 'OVERDUE' ELSE '' END AS deadline FROM log.incident_capa c JOIN log.incidents i ON i.incident_id=c.incident_id WHERE c.status IN ('OPEN','IN_PROGRESS') ORDER BY c.due_date ASC NULLS LAST LIMIT 200
```

GRC Masking Coverage

Active Masking Rules - *stat*

```
SELECT COUNT(*) FROM config.masking_rules WHERE is_active=TRUE
```

Sensitive Columns (total) - *stat*

```
SELECT COUNT(*) FROM monitoring.sensitive_schema
```

Unmasked Sensitive Columns - *stat*

```
SELECT COUNT(*) FROM config.v_unmasked_sensitive_columns
```

Token Vault Entries - *stat*

```
SELECT COUNT(*) FROM config.masking_tokens
```

Rules by Mask Type - *piechart*

```
SELECT mask_type, COUNT(*) AS rules FROM config.masking_rules WHERE is_active=TRUE GROUP BY mask_type ORDER BY rules DESC
```

Rules by PII Type - *piechart*

```
SELECT pii_type, COUNT(*) AS rules FROM config.masking_rules WHERE is_active=TRUE GROUP BY pii_type ORDER BY rules DESC
```

Active Rules by Regulation - *barchart*

```
SELECT COALESCE(regulation,'unclassified') AS regulation, COUNT(*) AS active_rules FROM config.masking_rules WHERE is_active=TRUE GROUP BY regulation ORDER BY active_rules DESC
```

Unmasked Sensitive Columns (Risk Gap) - *table*

```
SELECT server, database_name, table_name, column_name, pii_type, data_type FROM config.v_unmasked_sensitive_columns ORDER BY server, table_name
```

Active Masking Rules - *table*

```
SELECT server_name, table_name, column_name, pii_type, mask_type, regulation, array_to_string(allowed_roles, ', ') AS bypass_roles, updated_at FROM config.v_masking_rules_detail WHERE is_active=TRUE ORDER BY server_name, table_name, column_name
```

Masking Coverage Summary - *table*

```
SELECT server_name, regulation, total_sensitive_columns, actively_masked, array_to_string(pii_types_covered, ', ') AS pii_types FROM config.v_masking_coverage ORDER BY server_name, regulation
```

GRC Overview

BLOCKED (24h) - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE action_taken='BLOCKED' AND event_time >= NOW()-INTERVAL '24 hours'
```

ALERTED (24h) - *stat*

```
SELECT COUNT(*) FROM log.firewall_audit_log WHERE action_taken='ALERTED' AND event_time >= NOW()-INTERVAL '24 hours'
```

Open Incidents - *stat*

```
SELECT COUNT(*) FROM log.security_incidents WHERE status IN ('OPEN','ACKNOWLEDGED')
```

High-Risk Users (≥70) - *stat*

```
SELECT COUNT(*) FROM monitoring.user_risk_profiles WHERE risk_score >= 70
```

Active Blocked IPs - *stat*

```
SELECT COUNT(*) FROM config.blocked_ips WHERE is_active=TRUE AND (expires_at IS NULL OR expires_at > NOW())
```

Active Masking Rules - *stat*

```
SELECT COUNT(*) FROM config.masking_rules WHERE is_active=TRUE
```

Top Servers by Events (24h) - *barchart*

```
SELECT server_name, COUNT(*) AS events FROM log.firewall_audit_log WHERE event_time >= NOW()-INTERVAL '24 hours' GROUP BY server_name ORDER BY events DESC LIMIT 10
```

Firewall Events by Action (hourly) - *timeseries*

```
SELECT date_trunc('hour', event_time) AS "time", SUM(CASE WHEN action_taken='BLOCKED' THEN 1 ELSE 0 END) AS "BLOCKED", SUM(CASE WHEN action_taken='ALERTED' THEN 1 ELSE 0 END) AS "ALERTED", SUM(CASE WHEN action_taken='MASKED' THEN 1 ELSE 0 END) AS "MASKED", SUM(CASE WHEN action_taken='ALLOWED' THEN 1 ELSE 0 END) AS "ALLOWED" FROM log.firewall_audit_log WHERE $__timeFilter(event_time) GROUP BY 1 ORDER BY 1
```

Recent BLOCKED / CRITICAL Events - *table*

```
SELECT a.event_time, a.server_name, a.db_user, a.client_ip, a.action_taken, a.risk_score, a.regulation, p.policy_name FROM log.firewall_audit_log a LEFT JOIN config.firewall_policies p ON p.policy_id = a.matched_policy WHERE a.action_taken IN ('BLOCKED','ALERTED') AND a.risk_score >= 70 AND $__timeFilter(a.event_time) ORDER BY a.event_time DESC LIMIT 50
```

Exceptions Pending Approval - *stat*

```
SELECT COUNT(*) FROM config.policy_exceptions WHERE approval_status='PENDING'
```

Overdue Access Reviews - *stat*

```
SELECT COUNT(*) FROM log.access_review_instances WHERE status='OPEN' AND period_end < NOW()
```

Open Critical Incidents - *stat*

```
SELECT COUNT(*) FROM log.incidents WHERE severity='CRITICAL' AND state NOT IN ('CLOSED')
```

GDPR 72h Overdue - *stat*

```
SELECT COUNT(*) FROM log.incidents WHERE involves_personal_data=TRUE AND gdpr_notification_due<NOW() AND gdpr_notified_at IS NULL AND state!='CLOSED'
```

Pending Attestations - *stat*

```
SELECT COUNT(*) FROM log.attestations WHERE attestation_status='PENDING'
```

Transfers Without Legal Basis - *stat*

```
SELECT COUNT(*) FROM log.cross_border_transfers WHERE has_legal_basis=FALSE AND detected_at>=NOW()-INTERVAL '30 days'
```

Hash Chain Valid - *stat*

```
SELECT CASE WHEN COUNT(*)=0 THEN 0 WHEN bool_and(is_valid) THEN 1 ELSE 0 END FROM log.hash_chain_checkpoints WHERE verified_at>=NOW()-INTERVAL '24 hours'
```

GRC Policy Exceptions

Pending Approval - *stat*

```
SELECT COUNT(*) FROM config.policy_exceptions WHERE approval_status='PENDING'
```

Active Approved Exceptions - *stat*

```
SELECT COUNT(*) FROM config.v_active_policy_exceptions
```

Expiring in <7 Days - *stat*

```
SELECT COUNT(*) FROM config.policy_exceptions WHERE approval_status='APPROVED' AND expires_at BETWEEN NOW() AND NOW()+INTERVAL '7 days'
```

Rejected (30d) - *stat*

```
SELECT COUNT(*) FROM config.policy_exceptions WHERE approval_status='REJECTED' AND created_at >= NOW()-INTERVAL '30 days'
```

Exception Requests per Day - *timeseries*

```
SELECT date_trunc('day', created_at) AS "time", SUM(CASE WHEN approval_status='APPROVED' THEN 1 ELSE 0 END) AS "Approved", SUM(CASE WHEN approval_status='PENDING' THEN 1 ELSE 0 END) AS "Pending", SUM(CASE WHEN approval_status='REJECTED' THEN 1 ELSE 0 END) AS "Rejected" FROM config.policy_exceptions WHERE $__timeFilter(created_at) GROUP BY 1 ORDER BY 1
```

Exceptions by Status - *piechart*

```
SELECT approval_status, COUNT(*) AS count FROM config.policy_exceptions GROUP BY approval_status
```

Active Exceptions by Regulation - *barchart*

```
SELECT COALESCE(regulation,'-') AS regulation, COUNT(*) AS count FROM config.policy_exceptions WHERE
approval_status='APPROVED' AND (expires_at IS NULL OR expires_at > NOW()) GROUP BY regulation ORDER BY count DESC
```

Exceptions by Server - *barchart*

```
SELECT COALESCE(server_name,'any') AS server_name, COUNT(*) AS count FROM config.policy_exceptions WHERE
approval_status='APPROVED' AND (expires_at IS NULL OR expires_at > NOW()) GROUP BY server_name ORDER BY count DESC LIMIT 10
```

Top Requestors (90d) - *barchart*

```
SELECT created_by, COUNT(*) AS requests FROM config.policy_exceptions WHERE created_at >= NOW()-INTERVAL '90 days' GROUP BY
created_by ORDER BY requests DESC LIMIT 10
```

Pending Approval - Action Required - *table*

```
SELECT exception_id, COALESCE(p.policy_name,'-') AS policy, COALESCE(e.server_name,'any') AS server, COALESCE(e.db_user,'any')
AS db_user, COALESCE(e.client_ip_cidr,'any') AS client_ip_cidr, COALESCE(e.regulation,'-') AS regulation, e.reason,
e.created_by, e.created_at, e.expires_at FROM config.policy_exceptions e LEFT JOIN config.firewall_policies p ON p.policy_id =
e.policy_id WHERE e.approval_status='PENDING' ORDER BY e.created_at ASC LIMIT 200
```

Active Approved Exceptions - *table*

```
SELECT e.exception_id, COALESCE(p.policy_name,'-') AS policy, COALESCE(e.server_name,'any') AS server,
COALESCE(e.db_user,'any') AS db_user, COALESCE(e.client_ip_cidr,'any') AS client_ip_cidr, COALESCE(e.regulation,'-') AS
regulation, e.approved_by, e.approved_at, e.expires_at FROM config.v_active_policy_exceptions e LEFT JOIN
config.firewall_policies p ON p.policy_id = e.policy_id ORDER BY e.expires_at ASC NULLS LAST LIMIT 200
```

Exception History (All) - *table*

```
SELECT e.exception_id, COALESCE(p.policy_name,'-') AS policy, COALESCE(e.server_name,'any') AS server,
COALESCE(e.db_user,'any') AS db_user, e.approval_status, COALESCE(e.regulation,'-') AS regulation, e.created_by, e.created_at,
e.approved_by, e.approved_at, e.expires_at FROM config.policy_exceptions e LEFT JOIN config.firewall_policies p ON p.policy_id
= e.policy_id ORDER BY e.created_at DESC LIMIT 500
```

GRC Risk Register

Open Risks - *stat*

```
SELECT COUNT(*) FROM config.risk_register WHERE status='OPEN'
```

Critical / High Risks (Score ≥15) - *stat*

```
SELECT COUNT(*) FROM config.risk_register WHERE inherent_risk_score>=15 AND status NOT IN ('CLOSED','ACCEPTED')
```

In Treatment - *stat*

```
SELECT COUNT(*) FROM config.risk_register WHERE status='IN_TREATMENT'
```

Avg Residual Risk Score - *stat*

```
SELECT ROUND(AVG(residual_risk_score)::numeric,1) FROM config.risk_register WHERE status NOT IN ('CLOSED')
```

Risks by Inherent Score Bucket - *barchart*

```
SELECT CASE WHEN inherent_risk_score>=20 THEN 'Critical (20-25)' WHEN inherent_risk_score>=15 THEN 'High (15-19)' WHEN
inherent_risk_score>=10 THEN 'Medium (10-14)' WHEN inherent_risk_score>=5 THEN 'Low (5-9)' ELSE 'Minimal (<5)' END AS bucket,
COUNT(*) AS risks FROM config.risk_register WHERE status NOT IN ('CLOSED') GROUP BY bucket ORDER BY MIN(inherent_risk_score)
```

```
DESC
```

Risks by Category - *barchart*

```
SELECT COALESCE(risk_category,'other') AS category, COUNT(*) AS risks FROM config.risk_register WHERE status NOT IN ('CLOSED') GROUP BY category ORDER BY risks DESC
```

Risks by Treatment Strategy - *piechart*

```
SELECT COALESCE(treatment_strategy,'NONE') AS strategy, COUNT(*) AS risks FROM config.risk_register WHERE status NOT IN ('CLOSED') GROUP BY strategy
```

Top Assets by Residual Risk Score - *barchart*

```
SELECT asset_name, ROUND(SUM(residual_risk_score)::numeric,1) AS total_residual FROM config.risk_register WHERE status NOT IN ('CLOSED') GROUP BY asset_name ORDER BY total_residual DESC LIMIT 10
```

Risks by Regulation - *barchart*

```
SELECT UNNEST(COALESCE(regulation,ARRAY['-'])) AS reg, COUNT(*) AS risks FROM config.risk_register WHERE status NOT IN ('CLOSED') GROUP BY reg ORDER BY risks DESC
```

Risk Register - *table*

```
SELECT risk_id, asset_name, risk_title, risk_category, likelihood, impact, inherent_risk_score, control_effectiveness AS ctrl_eff, ROUND(residual_risk_score::numeric,1) AS residual, treatment_strategy, risk_owner, status, review_date FROM config.risk_register ORDER BY inherent_risk_score DESC NULLS LAST, risk_id LIMIT 500
```

GRC Risk Scoring

High-Risk Users (≥70) - *stat*

```
SELECT COUNT(*) FROM monitoring.user_risk_profiles WHERE risk_score >= 70
```

Critical-Risk Users (≥90) - *stat*

```
SELECT COUNT(*) FROM monitoring.user_risk_profiles WHERE risk_score >= 90
```

Average Risk Score - *gauge*

```
SELECT AVG(risk_score)::int AS value FROM monitoring.user_risk_profiles
```

Monitored Users - *stat*

```
SELECT COUNT(*) FROM monitoring.user_risk_profiles
```

Risk Score Trend (events) - *timeseries*

```
SELECT date_trunc('hour', event_time) AS "time", AVG(risk_score)::int AS "avg_score", MAX(risk_score) AS "max_score" FROM monitoring.user_risk_events WHERE $__timeFilter(event_time) GROUP BY 1 ORDER BY 1
```

Users by Risk Band - *barchart*

```
SELECT CASE WHEN risk_score >= 90 THEN '90-100 Critical' WHEN risk_score >= 70 THEN '70-89 High' WHEN risk_score >= 40 THEN '40-69 Medium' ELSE '0-39 Low' END AS risk_band, COUNT(*) AS users FROM monitoring.user_risk_profiles GROUP BY risk_band ORDER BY risk_band DESC
```

Top 30 High-Risk Users - *table*

```
SELECT server_name, risk_score, consecutive_high_risk, ROUND(avg_queries_per_hour::numeric, 1) AS avg_qph,
ROUND(avg_duration_secs::numeric, 2) AS avg_dur_s, CASE WHEN typical_hour_start IS NOT NULL THEN
typical_hour_start::text||':00-'||typical_hour_end::text||':00' ELSE '-' END AS typical_hours, last_updated FROM
monitoring.user_risk_profiles ORDER BY risk_score DESC LIMIT 30
```

Recent Risk Events - *table*

```
SELECT event_time, server_name, risk_score, risk_factors FROM monitoring.user_risk_events WHERE $__timeFilter(event_time)
ORDER BY event_time DESC LIMIT 100
```

GRC Threat Response

Response Success Rate (%) - *gauge*

```
SELECT CASE WHEN COUNT(*)=0 THEN 100 ELSE ROUND(100.0*SUM(CASE WHEN success THEN 1 ELSE 0 END)/COUNT(*), 1) END AS value FROM
log.threat_response_log WHERE triggered_at >= NOW()-INTERVAL '24 hours'
```

Open Incidents - *stat*

```
SELECT COUNT(*) FROM log.security_incidents WHERE status IN ('OPEN','ACKNOWLEDGED')
```

Blocked IPs (active) - *stat*

```
SELECT COUNT(*) FROM config.blocked_ips WHERE is_active=TRUE AND (expires_at IS NULL OR expires_at > NOW())
```

Suspended Users (active) - *stat*

```
SELECT COUNT(*) FROM config.suspended_users WHERE is_active=TRUE AND (expires_at IS NULL OR expires_at > NOW())
```

Responses (24h) - *stat*

```
SELECT COUNT(*) FROM log.threat_response_log WHERE triggered_at >= NOW()-INTERVAL '24 hours'
```

Active Suspended Users - *table*

```
SELECT server_name, login_name, reason, suspended_by, suspended_at, expires_at, remaining FROM config.v_suspended_users_active
ORDER BY suspended_at DESC
```

Automated Responses per Hour - *timeseries*

```
SELECT date_trunc('hour', triggered_at) AS "time", SUM(CASE WHEN response_type='BLOCK_IP' THEN 1 ELSE 0 END) AS "BLOCK_IP",
SUM(CASE WHEN response_type='SUSPEND_USER' THEN 1 ELSE 0 END) AS "SUSPEND_USER", SUM(CASE WHEN response_type='NOTIFY' THEN 1
ELSE 0 END) AS "NOTIFY", SUM(CASE WHEN response_type='ESCALATE' THEN 1 ELSE 0 END) AS "ESCALATE", SUM(CASE WHEN
response_type='WEBHOOK' THEN 1 ELSE 0 END) AS "WEBHOOK" FROM log.threat_response_log WHERE $__timeFilter(triggered_at) GROUP
BY 1 ORDER BY 1
```

Open Security Incidents - *table*

```
SELECT incident_id, title, severity, regulation, server_name, db_user, client_ip, status, created_at FROM log.v_active_threats
ORDER BY CASE severity WHEN 'CRITICAL' THEN 1 WHEN 'HIGH' THEN 2 ELSE 3 END, created_at DESC
```

Recent Automated Responses - *table*

```
SELECT triggered_at, playbook_name, trigger_type, subject, response_type, CASE WHEN success THEN 'OK' ELSE 'FAILED' END AS
result, COALESCE(error_message, (response_detail->>'detail')::text, '') AS detail FROM log.threat_response_log WHERE
$__timeFilter(triggered_at) ORDER BY triggered_at DESC LIMIT 50
```

Active Blocked IPs - *table*

```
SELECT ip_address, reason, blocked_by, blocked_at, expires_at, remaining FROM config.v_blocked_ips_active ORDER BY blocked_at
DESC
```

Active Response Playbooks - *table*

```
SELECT playbook_name, trigger_type, trigger_threshold, COALESCE(trigger_regulation,'any') AS regulation, response_type,
cooldown_mins FROM config.threat_response_playbooks WHERE is_active=TRUE ORDER BY playbook_id
```

GRC Workflow Segregation of Duties

SoD Violations (30d) - *stat*

```
SELECT COUNT(*) FROM log.sod_violations WHERE detected_at>=NOW()-INTERVAL '30 days'
```

Violations This Week - *stat*

```
SELECT COUNT(*) FROM log.sod_violations WHERE detected_at>=NOW()-INTERVAL '7 days'
```

Active SoD Rules - *stat*

```
SELECT COUNT(*) FROM config.sod_rules WHERE is_active=TRUE
```

Unique Violators (30d) - *stat*

```
SELECT COUNT(DISTINCT acknowledged_by) FROM log.sod_violations WHERE detected_at>=NOW()-INTERVAL '30 days'
```

Workflow SoD Violations per Day - *timeseries*

```
SELECT date_trunc('day', detected_at) AS "time", COUNT(*) AS violations FROM log.sod_violations WHERE
$__timeFilter(detected_at) GROUP BY 1 ORDER BY 1
```

Violations by Action Type (30d) - *barchart*

```
SELECT rule_name, COUNT(*) AS violations FROM log.sod_violations WHERE detected_at>=NOW()-INTERVAL '30 days' GROUP BY
rule_name ORDER BY violations DESC
```

Top Violators (30d) - *barchart*

```
--SELECT COUNT(*) FROM log.sod_violations WHERE detected_at>=NOW()-INTERVAL '30 days' SELECT acknowledged_by, COUNT(*) AS
attempts FROM log.sod_violations WHERE detected_at>=NOW()-INTERVAL '30 days' GROUP BY acknowledged_by ORDER BY attempts DESC
LIMIT 10
```

SoD Rule Configuration - *table*

```
SELECT rule_id, rule_name, description, CASE WHEN is_active THEN 'Active' ELSE 'Disabled' END AS status FROM config.sod_rules
ORDER BY rule_id
```

Workflow SoD Violation Log (30d) - *table*

```
SELECT v.detected_at, v.rule_name, v.attempted_by, v.resource_id, v.resource_owner, v.violation_reason FROM log.sod_violations
v WHERE v.acknowledged_at>=NOW()-INTERVAL '30 days' ORDER BY v.detected_at DESC LIMIT 300
```

IPS Dashboard - FortiAnalyzer

Total Events - *stat*

```
SELECT COALESCE(count(*),0) AS "Total Events" FROM alerts.alert_log where $__timeFilter(entry_date)
```

Critical - *stat*

```
SELECT count(*) FROM alerts.alert_log WHERE risk_level ='critical' and $__timeFilter(entry_date)
```

High - *stat*

```
SELECT count(*) FROM alerts.alert_log WHERE risk_level ='high' and $__timeFilter(entry_date)
```

Medium - *stat*

```
SELECT count(*) FROM alerts.alert_log WHERE risk_level ='medium' and $__timeFilter(entry_date)
```

Blocked - *stat*

```
SELECT COALESCE(SUM(count),0) AS "Blocked" FROM siem.fortianalyzer_ips_events WHERE action='blocked'
```

Monitored - *stat*

```
select count(*) from alerts.mail_alert_log where recipients is not null and $__timeFilter(entry_date)
```

Intrusions by Severity - *piechart*

```
SELECT risk_level AS "Severity", count(*) AS "Count" FROM alerts.alert_log GROUP BY risk_level ORDER BY CASE risk_level WHEN
'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 WHEN 'low' THEN 4 ELSE 5 END
```

Intrusions by Type - *barchart*

```
select RC.issue_name as "Type" , count(*) AS "Count" from alerts.alert_log al join rootcause.v_rootcauses rc on
rc.root_cause_id = al.root_cause_id where $__timeFilter(entry_date) group by al.root_cause_id , issue_name order by count(*)
desc limit 20
```

Intrusion Events Timeline (Last 7 Days) - *timeseries*

```
SELECT date_trunc('hour', entry_date)::timestamp AS "time" , COUNT(*) AS "count", risk_level AS severity FROM alerts.alert_log
where $__timeFilter(entry_date) GROUP BY risk_level, date_trunc('hour', entry_date) ORDER BY time;
```

Monitored Intrusions - *table*

```
SELECT root_cause_name AS "Attack", issue_name AS "Type", count(*) AS "Count" FROM alerts.alert_log al join
rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where $__timeFilter(entry_date) group by root_cause_name ,
issue_name
```

Top Victims - *table*

```
SELECT server , root_cause_name AS "Attack", issue_name AS "Type", count(*) AS "Count" FROM alerts.mail_alert_log al join
rootcause.v_rootcauses rc on rc.root_cause_id = al.metric_name group by root_cause_name , issue_name ,server
```

Blocked Intrusions - *table*

```
SELECT root_cause_name AS "Attack", issue_name AS "Type", risk_level AS "Severity", count(*) AS "Count" FROM
alerts.mail_alert_log ma join rootcause.v_rootcauses rc on rc.root_cause_id = ma.metric_name where $__timeFilter(entry_date)
group by root_cause_name , issue_name ,risk_level ORDER BY CASE risk_level WHEN 'critical' THEN 1 WHEN 'high' THEN 2 WHEN
'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Top Attack Sources - *table*

```
SELECT count(*) *100 / (select count(*) from alerts.alert_log al1 where $__timeFilter(al1.entry_date) ) pct_of_total ,
gmmr.metric_metadata , risk_level FROM alerts.alert_log al join monitoring.general_metric_metadata_results gmmr on gmmr.server
= al.server and al.root_cause_id = gmmr.metric_name and al.entry_date between gmmr.entry_date -interval '1 second' and
gmmr.entry_date +interval '1 second' where $__timeFilter(al.entry_date) group by risk_level , metric_metadata order by
pct_of_total desc
```

Linked servers / DBLINKS - Hidden credentials

Linked server / DBLINKS Hidden credentials - *gauge*

```
select count(*) "Number of hidden credentials", server from monitoring.linked_servers_hidden_credentials group by server
```

(untitled panel) - *table*

```
select distinct server from monitoring.linked_servers_hidden_credentials
```

Active users - *bargauge*

```
select sum("Number of connections") from ( select count(*) "Number of connections" from monitoring.tcp_connections where
connect_time between Now() - interval '1 minute' and Now() union all select count(*) from monitoring.v_oracle_transactions
where entry_date > Now() - interval '1 minute' )
```

Recent alerts - *stat*

```
select (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and
query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR
1=1%', '%AND 1=1%' , '%OR ''a''=''a%''])) as "Boolean-based injections" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as
"union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval
'1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*)
from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.active_transactions
where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as
"shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now()
- interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection" from
monitoring.active_transactions group by server
```

Linked server / DBLINKS Hidden credentials - *table*

```
select * from monitoring.linked_servers_hidden_credentials
```

Connection number per user - *barchart*

```
select sum(connection_count::int) , server ||'-'||login_name from monitoring.connection_count group by server
||'-'||login_name
```

Active threats - *gauge*

```
select count(*) "suspicious queries" from monitoring.v_suspiscous
```

Locked accounts

Locked accounts - *barchart*

```
select server , count(*) lock_date from monitoring.v_oracle_locked_accounts group by server
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_oracle_locked_accounts
```

Locked accounts - *table*

```
select server , username , account_status , to_timestamp(lock_date::bigint / 1000) lock_date from
monitoring.v_oracle_locked_accounts
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

Mail

Mail configuration - *table*

```
select mc.* , mg.recipients from config.mail_config mc join config.mail_groups mg on mg.mail_config_id = mc.row_id
```

PII - Sensitive columns

Total Events - *stat*

```
SELECT COALESCE(count(*),0) AS "Total Events" FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id =
al.root_cause_id where $__timeFilter(entry_date) and rc.root_cause_desc like '%PPL%'
```

Critical - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='critical' and $__timeFilter(entry_date)
```

High - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='high' and $__timeFilter(entry_date)
```

Medium - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='medium' and $__timeFilter(entry_date)
```

Blcked - *stat*

```
SELECT count(*) FROM alerts.mail_alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.metric_name where
rc.root_cause_desc like '%PPL%' and $__timeFilter(entry_date)
```

Monitored - *stat*

```
select count(*) from alerts.mail_alert_log where recipients is not null and $__timeFilter(entry_date)
```

Root causes - *table*

```
select root_cause_ID ,step_name,ROOT_CAUSE_DESC , risk_level from ( select distinct rc.root_cause_ID ,rc.step_name,
rc.ROOT_CAUSE_DESC , al.risk_level FROM rootcause.v_rootcauses rc join alerts.alert_log al on al.root_cause_id =
rc.root_cause_ID where rc.root_cause_id like 'SEC-SQL-PRI%' and vendor_name = 'sqlserver' ) ORDER BY CASE risk_level WHEN
'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Detailed findings - *table*

```
SELECT * FROM monitoring.get_alert_log_resultset('${root_cause_id}', $__timeFrom()) WHERE '${root_cause_id}' <> '' LIMIT 10;
```

Root cause details - ${root*cause*id} - *table*

```
SELECT j.value AS result FROM jsonb_array_elements(
COALESCE(monitoring.get_root_cause_resultset(NULLIF('${root_cause_id}',''), $__timeFrom())::jsonb, '[]'::jsonb) ) AS j LIMIT
500
```

PII - Sensitive data in use

Total Events - *stat*

```
SELECT COALESCE(count(*),0) AS "Total Events" FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id =
al.root_cause_id where $__timeFilter(entry_date) and rc.root_cause_desc like '%PPL%'
```

Critical - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='critical' and $__timeFilter(entry_date)
```

High - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='high' and $__timeFilter(entry_date)
```

Medium - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='medium' and $__timeFilter(entry_date)
```

Blcked - *stat*

```
SELECT count(*) FROM alerts.mail_alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.metric_name where
rc.root_cause_desc like '%PPL%' and $__timeFilter(entry_date)
```

Monitored - *stat*

```
select count(*) from alerts.mail_alert_log where recipients is not null and $__timeFilter(entry_date)
```

Root causes - *table*

```
select root_cause_ID ,step_name,ROOT_CAUSE_DESC , risk_level from ( select distinct rc.root_cause_ID ,rc.step_name,
rc.ROOT_CAUSE_DESC , al.risk_level FROM rootcause.v_rootcauses rc join alerts.alert_log al on al.root_cause_id =
rc.root_cause_ID where rc.root_cause_id like 'SEC-SQL-PRI%' and vendor_name = 'sqlserver' ) ORDER BY CASE risk_level WHEN
'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Detailed findings - *table*

```
SELECT * FROM monitoring.get_alert_log_resultset('${root_cause_id}', $__timeFrom()) WHERE '${root_cause_id}' <> '' LIMIT 10;
```

Root cause details - ${root*cause*id} - *table*

```
SELECT j.value AS result FROM jsonb_array_elements(
COALESCE(monitoring.get_root_cause_resultset(NULLIF('${root_cause_id}',''), $__timeFrom())::jsonb, '[]'::jsonb) ) AS j LIMIT
```

```
500
```

PPL- Protection of Privacy Law, 5741 - 1981

Total Events - *stat*

```
SELECT COALESCE(count(*),0) AS "Total Events" FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id =
al.root_cause_id where $__timeFilter(entry_date) and rc.root_cause_desc like '%PPL%'
```

Critical - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='critical' and $__timeFilter(entry_date)
```

High - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='high' and $__timeFilter(entry_date)
```

Medium - *stat*

```
SELECT count(*) FROM alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where
rc.root_cause_desc like '%PPL%' and al.risk_level ='medium' and $__timeFilter(entry_date)
```

Blcked - *stat*

```
SELECT count(*) FROM alerts.mail_alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.metric_name where
rc.root_cause_desc like '%PPL%' and $__timeFilter(entry_date)
```

Monitored - *stat*

```
select count(*) from alerts.mail_alert_log where recipients is not null and $__timeFilter(entry_date)
```

Intrusions by Severity - *piechart*

```
SELECT risk_level AS "Severity", count(*) AS "Count" FROM alerts.alert_log GROUP BY risk_level ORDER BY CASE risk_level WHEN
'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 WHEN 'low' THEN 4 ELSE 5 END
```

Intrusions by Type - *barchart*

```
select RC.issue_name as "Type" , count(*) AS "Count" from alerts.alert_log al join rootcause.v_rootcauses rc on
rc.root_cause_id = al.root_cause_id where $__timeFilter(entry_date) and rc.root_cause_desc like '%PPL%' group by
al.root_cause_id , issue_name
```

Intrusion Events Timeline (Last 7 Days) - *timeseries*

```
SELECT date_trunc('hour', entry_date)::timestamp AS "time" , COUNT(*) AS "count", rc.risk_level AS severity FROM
alerts.alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where $__timeFilter(entry_date) and
root_cause_desc like '%PPL%' GROUP BY rc.risk_level, date_trunc('hour', entry_date) ORDER BY time;
```

Blocked Intrusions - *table*

```
SELECT root_cause_name AS "Attack", issue_name AS "Type", risk_level AS "Severity", count(*) AS "Count" FROM
alerts.mail_alert_log ma join rootcause.v_rootcauses rc on rc.root_cause_id = ma.metric_name where $__timeFilter(entry_date)
and rc.root_cause_DESC like '%PPL%' group by root_cause_name , issue_name ,risk_level ORDER BY CASE risk_level WHEN 'critical'
THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Monitored Intrusions - *table*

```
SELECT root_cause_name AS "Attack", issue_name AS "Type", count(*) AS "Count" , rc.root_cause_desc FROM alerts.alert_log al
join rootcause.v_rootcauses rc on rc.root_cause_id = al.root_cause_id where $__timeFilter(entry_date) and rc.root_cause_desc
```

```
like '%PPL%' group by root_cause_name , issue_name , rc.root_cause_desc
```

Top Victims - *table*

```
SELECT server , root_cause_name AS "Attack", issue_name AS "Type", count(*) AS "Count" FROM alerts.mail_alert_log al join rootcause.v_rootcauses rc on rc.root_cause_id = al.metric_name where root_cause_desc like '%PPL%' group by root_cause_name , issue_name ,server
```

Root causes - *table*

```
select rc.root_cause_ID , rc.ROOT_CAUSE_DESC FROM rootcause.v_rootcauses rc where rc.root_cause_DESC like '%PPL%' and vendor_name = 'sqlserver' ORDER BY CASE risk_level WHEN 'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Top Attack Sources - *table*

```
SELECT count(*) *100 / (select count(*) from alerts.alert_log al1 where $__timeFilter(al1.entry_date) ) pct_of_total , gmmr.metric_metadata , risk_level FROM alerts.alert_log al join monitoring.general_metric_metadata_results gmmr on gmmr.server = al.server and al.root_cause_id = gmmr.metric_name and al.entry_date between gmmr.entry_date -interval '1 second' and gmmr.entry_date +interval '1 second' where $__timeFilter(al.entry_date) group by risk_level , metric_metadata order by pct_of_total desc
```

Policy not enforced

Policy not enforced - *barchart*

```
select server , count(*) lock_date from monitoring.v_policy_enforeced group by server
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_policy_enforeced
```

Policy not enforced - *table*

```
select * from monitoring.v_policy_enforeced
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server , (sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->> 'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time = TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS "union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by server
```

Processes

Processes - *table*

```
select process_name , is_active as Active , interval from metrics.registered_processes
```

```
select SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) + SUM(CASE WHEN sleep_time = TRUE THEN 1 ELSE 0 END) +
SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN long_literal = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN
outfile_copy = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN schema_access =
TRUE THEN 1 ELSE 0 END) + SUM(CASE WHEN stacked_query = TRUE THEN 1 ELSE 0 END) + SUM(CASE WHEN commented_payload = TRUE THEN
1 ELSE 0 END) + SUM(CASE WHEN sensitive_columns = TRUE THEN 1 ELSE 0 END) "Active threats" FROM
monitoring.v_sql_feature_predictions
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

Connectivity reports - *barchart*

```
select count(*) , server ,client_net_address from monitoring.tcp_connections where connect_Time > Now() - interval '1 hour'
group by server ,client_net_address
```

Recent alerts

Security dashboard - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Critical issues - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Active threats - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Active threats - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Active users - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Cyber attacks - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Recent alerts - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Sensitivity reports - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

SQL Injection analysis - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Data activity monitoring - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Connectivity reports - *table*

```
SELECT tcp.server ,tcp.client_net_address, at.query , at.login_name , tcp.connect_time FROM monitoring.TCP_connections tcp
left outer join monitoring.active_transactions at on at.server = tcp.server and at.session_id = tcp.session_id and
at.start_time between tcp.connect_time and at.last_request_end_time WHERE connect_time between Now() - interval '5 minutes'
and now()
```

Retention

Retention Overview - *table*

```
SELECT ro.category, ro.item, rc.root_cause_name, ro.data_size, ro.data_bytes, ro.row_count, ro.space_free,
dm.retention_months, CASE WHEN dm.retention_months IS NOT NULL THEN '' END AS up, CASE WHEN dm.retention_months IS NOT NULL
THEN '' END AS down, (dm.retention_months + 1) AS up_val, GREATEST(dm.retention_months - 1, 0) AS down_val, CASE WHEN ro.item
IS NOT NULL THEN ' Dump' END AS dump FROM metrics.t_retention_overview ro LEFT JOIN (SELECT DISTINCT root_cause_id,
root_cause_name FROM rootcause.v_rootcauses) rc ON rc.root_cause_id = ro.item LEFT JOIN config.dump_metrics dm ON
dm.metric_name = ro.item ORDER BY ro.category, ro.item
```

Apply retention edit (auto-runs on / click) - *table*

```
UPDATE config.dump_metrics SET retention_months = NULLIF('${edit_value}',''):::int WHERE metric_name =
NULLIF('${edit_metric}','') RETURNING metric_name AS edited_metric, retention_months AS new_retention
```

Current dump location - *table*

```
SELECT config.get_dump_location() AS dump_location
```

Save dump location (refresh to save) - *table*

```
SELECT CASE WHEN '${dump_location}' <> '' AND '${dump_location}' <> coalesce(config.get_dump_location(),'') THEN
config.set_dump_location('${dump_location}') ELSE config.get_dump_location() END AS dump_location
```

Dump metrics (config.dump_metrics) - *table*

```
SELECT row_id, metric_name, retention_months, entry_date FROM config.dump_metrics ORDER BY metric_name
```

Dumps catalog (config.dumps) - *table*

```
SELECT row_id, metric_name, month, year, row_count, dump_location, entry_date, '■ Restore' AS restore FROM config.dumps ORDER
BY year DESC, month DESC, metric_name
```

Apply dump (auto-runs on Dump click) - *table*

```
SELECT * FROM config.dump_last_month(NULLIF('${dump_metric}',''))
```

Apply restore (auto-runs on Restore click) - *table*

```
SELECT * FROM config.restore_dump(NULLIF('${restore_id}','')::bigint)
```

Retention policy

Retention policy - *table*

```
select row_id , server , table_name , days_to_keep from config.retention_policy
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

Connectivity reports - *barchart*

```
select count(*) , server ,client_net_address from monitoring.tcp_connections where connect_Time > Now() - interval '1 hour'
group by server ,client_net_address
```

Rootcauses

Issues (click 'select' to load its root causes) - *table*

```
SELECT i.issue_id, i.name AS issue_name, left(i.description,220) AS issue_desc, (ARRAY['low','medium','high','critical'])[
max(CASE vr.risk_level WHEN 'low' THEN 1 WHEN 'medium' THEN 2 WHEN 'high' THEN 3 WHEN 'critical' THEN 4 ELSE 0 END) ] AS
risk_level, bool_or(vr.is_active) AS is_active, 'select ' AS open FROM rootcause.issues i LEFT JOIN rootcause.v_rootcauses vr
ON vr.issue_id = i.issue_id where i.domain_code = 'SEC' and i.database_type_code = 'SQL' GROUP BY i.issue_id, i.name,
i.description ORDER BY i.issue_id
```

Root causes for selected issue - toggle active / set risk level (saved via API) - *table*

```
WITH vr AS ( SELECT root_cause_id, bool_or(is_active) AS is_active, max(CASE risk_level WHEN 'low' THEN 1 WHEN 'medium' THEN 2
WHEN 'high' THEN 3 WHEN 'critical' THEN 4 ELSE 0 END) AS risk_ord FROM rootcause.v_rootcauses WHERE issue_id = '${sel_issue}'
GROUP BY root_cause_id ) SELECT rc.root_cause_id, rc.name AS root_cause_name, left(rc.description,220) AS root_cause_desc,
coalesce(vr.is_active,false) AS is_active, (ARRAY['low','medium','high','critical'])[vr.risk_ord] AS risk_level, rc.issue_id,
(NOT coalesce(vr.is_active,false))::text AS next_active, CASE WHEN coalesce(vr.is_active,false) THEN ' Disable' ELSE ' Enable'
END AS toggle, 'Low' AS set_low, 'Medium' AS set_medium, 'High' AS set_high, 'Critical' AS set_critical FROM
rootcause.root_causes rc LEFT JOIN vr ON vr.root_cause_id = rc.root_cause_id WHERE rc.issue_id = '${sel_issue}' ORDER BY
rc.root_cause_id
```

SIEM configuration

SIEM - *table*

```
select * from config.sime_interface
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

Connectivity reports - *barchart*

```
select count(*) , server ,client_net_address from monitoring.tcp_connections where connect_Time > Now() - interval '1 hour'
group by server ,client_net_address
```

SQL Injection

SQL injection by patterns - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
```

```
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

SQL Injections - Injection prevention - *table*

```
select root_cause , description from rootcause.v_root_causes_with_context
```

SQL injection - Boolean-based injections

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%information_schema.tables%', '%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

comment based injections - *table*

```
select SERVER , COLUMNS ,TABLES ,CONDITION , FUNC AS FUCTION , JOINS , LITERAL , QUERY from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND 1=1%' , '%OR ''a''=''a%''''])
```

SQL injection - Comment based

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%', '%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

comment based injections - *table*

```
select server, to_timestamp(creation_time::bigint/1000) "end time " , '(--.*?$|/\*.*?\*/)' "used to truncate the query" , QUERY from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )
```

SQL injection - Information schema probing

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%OR 1=1%',
'%AND 1=1%' , '%OR ''a''=''a%'''])) as "Boolean-based injections" , (select count(*) from monitoring.v_new_queries_last_hour
where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as
"union_based_injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%; DROP TABLE
users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.v_new_queries_last_hour where query
ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.v_new_queries_last_hour
where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*)
from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY
injection"
```

Information schema probing - *table*

```
select distinct server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])
```

SQL injection - Stacked queries

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%OR 1=1%',
'%AND 1=1%' , '%OR ''a''=''a%'''])) as "Boolean-based injections" , (select count(*) from monitoring.v_new_queries_last_hour
where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as
"union_based_injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%; DROP TABLE
users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.v_new_queries_last_hour where query
ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.v_new_queries_last_hour
where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*)
from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY
injection"
```

Stacked query injections - *table*

```
select distinct server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE%', '%; EXEC xp_cmdshell%'])
```

SQL injection - Time based

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%OR 1=1%',
'%AND 1=1%' , '%OR ''a''=''a%'''])) as "Boolean-based injections" , (select count(*) from monitoring.v_new_queries_last_hour
where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as
"union_based_injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%; DROP TABLE
users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.v_new_queries_last_hour where query
ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY
```

```
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.v_new_queries_last_hour where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Stacked query injections - *table*

```
select distinct server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE%', '%; EXEC xp_cmdshell%'])
```

SQL injection - Time based blind injection

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) ) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%', '%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Time based blind injections - *table*

```
select server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM monitoring.expensive_queries where query ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])
```

SQL injection - Union based

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND 1=1%' , '%OR ''a''=''a%''']) ) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%', '%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Union based injections - *table*

```
select server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])
```

SQL injection - tautology with comments

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

comment based injections - *table*

```
select server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM monitoring.expensive_queries
where query ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])
```

SQL injection -Authentication Bypass

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Authentication bypass - *table*

```
select server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM monitoring.expensive_queries
where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])
```

SQL injection -Encoding

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Encoding injection - *table*

```
select server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM monitoring.expensive_queries
where query ILIKE ANY (ARRAY['%0x414243%', '%CHAR(65,66,67)%'])
```

SQL injection -Error based injection

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Error based injections - *table*

```
select distinct server , to_timestamp(creation_time::bigint/1000) , query from monitoring.expensive_queries where query ILIKE
ANY (ARRAY['%CONVERT%', '%CAST%'])
```

SQL injection -Shell function

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Shell commands - *table*

```
select server , query , to_timeStamp(creation_time::bigint/1000) , avg_elapsed_time duration FROM monitoring.expensive_queries
where query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])
```

SQL injection -order / Group by

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''']) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
```

```
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

Order / Group by - *table*

```
select distinct server , query , to_timeStamp(creation_time::bigint/1000) last_request_end_time , avg_elapsed_time duration
FROM monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%2'])
```

SQL injection board

Critical issues - SQL injection - *gauge*

```
select (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as
"comment_based_injections", (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%OR 1=1%', '%AND
1=1%' , '%OR ''a''=''a%''])) as "Boolean-based injections" , (select count(*) from monitoring.expensive_queries where query
ILIKE ANY (ARRAY['%'' OR 1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as "union_based_injection"
, (select count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC
xp_cmdshell%'])) as "stacked_queries" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select
count(*) from monitoring.expensive_queries where query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%'' OR ''='%', '%'' OR ''x''='x''%', '%admin''--%'])) as
"Authentication bypass patterns" , (select count(*) from monitoring.expensive_queries where query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.expensive_queries where
query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as "shell_system_functions injection" , (select count(*) from
monitoring.expensive_queries where query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection"
```

SQL injection - Sleep time - *table*

```
select sfp."server" , sfp.query , at.last_request_end_time , command , logical_reads , reads , writes , login_name ,
program_name from monitoring.v_sql_feature_predictions sfp join monitoring.active_transactions at on at.query = sfp.query
where sfp.sleep_time is true limit 100;
```

Same login active from multiple hosts

Root causes - *table*

```
select root_cause_ID ,step_name,ROOT_CAUSE_DESC , risk_level from ( select distinct rc.root_cause_ID ,rc.step_name,
rc.ROOT_CAUSE_DESC , al.risk_level FROM rootcause.v_rootcauses rc join alerts.alert_log al on al.root_cause_id =
rc.root_cause_ID where rc.root_cause_id like 'SEC-SQL-ACC-010-RC10' and vendor_name = 'sqlserver' ) ORDER BY CASE risk_level
WHEN 'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Detailed findings - *table*

```
select * from monitoring.v_sec_sql_acc_010_rc10 where $__timeFilter(entry_date) and login_name !='dbdome_mon_usr'
```

Scan jobs for leaks

Jobs with Leaks - *stat*

```
SELECT COUNT(*) FROM monitoring.v_scan_jobs_for_leak
```

Servers Affected - *stat*

```
SELECT COUNT(DISTINCT server) FROM monitoring.v_scan_jobs_for_leak
```

Security Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Alerts Sent - *stat*

```
SELECT COUNT(*) FROM alerts.mail_alert_log WHERE $__timeFilter(entry_date)
```

Leaked Jobs by Server - *barchart*

```
SELECT server, COUNT(*) AS leaks FROM monitoring.v_scan_jobs_for_leak GROUP BY server ORDER BY leaks DESC
```

Sensitivity Issues by Server - *bargauge*

```
SELECT server, COUNT(*) AS issues FROM monitoring.sql_feature_predictions WHERE $__timeFilter(entry_date) GROUP BY server
ORDER BY issues DESC LIMIT 10
```

Scanned Jobs for Leaks - *table*

```
SELECT * FROM monitoring.v_scan_jobs_for_leak
```

SQL Injection Analysis - *barchart*

```
SELECT COALESCE(rc.name, gm.metric_name) AS type, COUNT(*) AS count FROM monitoring.general_metric_metadata_results gm LEFT
JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name WHERE gm.metric_name LIKE 'SEC-SQL-INJ%%' AND
(gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY type ORDER BY
count DESC LIMIT 10
```

Data Activity (Transactions/hour) - *barchart*

```
SELECT server, COUNT(*) AS transactions FROM monitoring.active_transactions WHERE last_request_end_time > NOW() - INTERVAL '1
hour' GROUP BY server ORDER BY transactions DESC
```

Security Issues Explorer

Security hierarchy - click a root cause to filter alerts - *table*

```
SELECT COALESCE( (SELECT ds.expected->>'severity' FROM rootcause.detection_paths dp JOIN rootcause.detection_path_steps dps ON
dps.detection_path_id = dp.id AND dps.sequence = 1 JOIN rootcause.detection_steps ds ON ds.id = dps.detection_step_id WHERE
dp.root_cause_id = rc.root_cause_id AND dp.is_active = true LIMIT 1), 'medium' ) AS risk_level, d.name AS domain_name, a.name
AS area_name, i.name AS issue_name, rc.root_cause_id, rc.name AS root_cause_name, rc.description AS root_cause_desc FROM
rootcause.root_causes rc JOIN rootcause.issues i ON i.issue_id = rc.issue_id JOIN rootcause.areas a ON a.code = i.area_code
AND a.database_type_code = i.database_type_code JOIN rootcause.domains d ON d.code = i.domain_code WHERE d.code = 'SEC' ORDER
BY risk_level, domain_name, area_name, issue_name, rc.name
```

Alerts for selected root cause - ${rc} - *table*

```
SELECT mal.entry_date, mal.server, mal.metric_name AS root_cause_id, rc.name AS root_cause_name, mal.transaction_type,
mal.subject, mal.recipients FROM alerts.mail_alert_log mal LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id =
mal.metric_name WHERE ('${rc:raw}' = '' OR mal.metric_name = '${rc:raw}') ORDER BY mal.entry_date DESC LIMIT 500
```

Send report by mail

(untitled panel) - *table*

```
select distinct server from monitoring.v_combined_transactions
```

(untitled panel) - *table*

```
select distinct TO_CHAR(last_request_end_time, 'YYYY-MM-DD') AS "start time" from monitoring.v_combined_transactions order by TO_CHAR(last_request_end_time, 'YYYY-MM-DD')
```

(untitled panel) - *table*

```
select distinct TO_CHAR(last_request_end_time, 'YYYY-MM-DD') AS "end time" from monitoring.v_combined_transactions
```

Sensitive Columns Explorer

Servers (sensitive columns) - *table*

```
SELECT server, count(DISTINCT db_name) AS databases, count(*) AS sensitive_columns FROM monitoring.v_sec_sql_pri_001_rc12 GROUP BY server ORDER BY server
```

Databases on ${srv} - *table*

```
SELECT db_name, count(DISTINCT table_schema||'.'||table_name) AS tables, count(*) AS columns FROM monitoring.v_sec_sql_pri_001_rc12 WHERE (NULLIF('${srv}','') IS NULL OR server='${srv}') GROUP BY db_name ORDER BY db_name
```

Sensitive columns - ${srv} / ${db} - *table*

```
SELECT distinct server, db_name, table_schema, table_name, column_name, data_type, pii_category, max_length, '+ track' AS track, ' mask' AS mask, ' encrypt' AS encrypt, '■ revert' AS revert FROM monitoring.v_sec_sql_pri_001_rc12 WHERE (NULLIF('${srv}','') IS NULL OR server='${srv}') AND (NULLIF('${db}','') IS NULL OR db_name='${db}') ORDER BY db_name, table_schema, table_name, column_name LIMIT 2000
```

Column details - ${dtbl}.${dcol} - *table*

```
SELECT j.value AS result FROM jsonb_array_elements(COALESCE(monitoring.get_root_cause_resultset('SEC-SQL-PRI-001-RC12', $__timeFrom())::jsonb,'[]'::jsonb)) j WHERE (NULLIF('${srv}','') IS NULL OR j.value->>'server'='${srv}') AND (NULLIF('${db}','') IS NULL OR j.value->>'db_name'='${db}') AND (NULLIF('${dsch}','') IS NULL OR j.value->>'schema_name'='${dsch}') AND (NULLIF('${dtbl}','') IS NULL OR j.value->>'table_name'='${dtbl}') AND (NULLIF('${dcol}','') IS NULL OR j.value->>'column_name'='${dcol}') LIMIT 500
```

Tracked sensitive columns (metrics.sensitive_columns) - *table*

```
SELECT server, db_name, schema_name, table_name, column_name, pii_category, entry_date AT TIME ZONE 'Asia/Jerusalem' AS entry_date FROM metrics.sensitive_columns ORDER BY entry_date DESC LIMIT 500
```

Masking activity (metrics.masking_log) - *table*

```
SELECT entry_date AT TIME ZONE 'Asia/Jerusalem' AS time, server, db_name, schema_name||'.'||table_name||'.'||column_name AS column, mask_function, action, detail FROM metrics.masking_log WHERE (NULLIF('${srv}','') IS NULL OR server='${srv}') ORDER BY log_id DESC LIMIT 200
```

Masked data preview - real vs masked (as test user) - *table*

```
SELECT server, db_name, schema_name||'.'||table_name AS "table", column_name AS "column", row_no, real_value AS "real (privileged)", masked_value AS "masked (test user)" FROM metrics.mask_preview WHERE (NULLIF('${srv}','') IS NULL OR server='${srv}') AND (NULLIF('${db}','') IS NULL OR db_name='${db}') ORDER BY entry_date DESC, schema_name, table_name, column_name, row_no LIMIT 500
```

Print (filtered view) - *table*

```
SELECT ' Print this filtered view (opens clean tab -> Ctrl+P / Save as PDF)' AS print
```

Encryption activity (metrics.encryption_log) - *table*

```
SELECT entry_date AT TIME ZONE 'Asia/Jerusalem' AS time, server, db_name, schema_name||'.'||table_name||coalesce('.'||nullif(column_name,''),'') AS object, action, detail FROM metrics.encryption_log
```

```
WHERE (NULLIF('${srv}','') IS NULL OR server='${srv}') ORDER BY log_id DESC LIMIT 200
```

Tokenized data preview - original vs token - *table*

```
SELECT schema_name||'.'||table_name AS "table", column_name AS "column", row_no, original_value, token_value FROM
metrics.encryption_preview WHERE (NULLIF('${srv}','') IS NULL OR server='${srv}') AND (NULLIF('${db}','') IS NULL OR
db_name='${db}') ORDER BY entry_date DESC, table_name, column_name, row_no LIMIT 500
```

Sensitivity report

Sensitive data (PII) - *table*

```
select at.server , at.query , last_request_end_time, c.column_name , c.table_name from monitoring.v_combined_transactions at
join( select TABLE_NAME , column_name from monitoring.v_sensitive_columns ) c on at.query like '%'||c.column_name ||'%' and
last_request_end_time > Now() - interval '15 minutes'
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_combined_transactions
```

(untitled panel) - *table*

```
select distinct TO_CHAR(last_request_end_time, 'YYYY-MM-DD') AS "start time" from monitoring.v_combined_transactions order by
TO_CHAR(last_request_end_time, 'YYYY-MM-DD')
```

(untitled panel) - *table*

```
select distinct TO_CHAR(last_request_end_time, 'YYYY-MM-DD') AS "end time" from monitoring.v_combined_transactions
```

Sensitive columns and schema (PII) - *table*

```
select * from monitoring.v_sensitive_columns
```

```
select server , tables , columns , query from ( select row_number() over ( partition by query order by p.entry_date desc ) seq
, p.server , susp."tables" , susp."columns" , susp.query from monitoring.sql_suspicious susp join monitoring.sensitive_schema
sc on susp.query like '%'||sc.column_name ||'%' join monitoring.sql_feature_predictions p on p.query_id = susp.query_id ) a
where seq=1
```

```
select server , table_name , column_name ,columns , query from ( select row_number() over ( partition by query order by
p.entry_date desc ) seq , p.server , sc.table_name , sc.column_name ,columns , p.condition , joins , func , p.query from
monitoring.metric_query_parsing p join monitoring.sensitive_schema sc on p.query like '%'||sc.column_name ||'%' ) where seq =
1
```

Stored procedure slower than its average

Root causes - *table*

```
select root_cause_ID ,step_name,ROOT_CAUSE_DESC , risk_level from ( select distinct rc.root_cause_ID ,rc.step_name,
rc.ROOT_CAUSE_DESC , al.risk_level FROM rootcause.v_rootcauses rc join alerts.alert_log al on al.root_cause_id =
rc.root_cause_ID where rc.root_cause_id like 'SEC-SQL-ACC-011%' and vendor_name = 'sqlserver' ) ORDER BY CASE risk_level WHEN
'critical' THEN 1 WHEN 'high' THEN 2 WHEN 'medium' THEN 3 ELSE 4 END DESC LIMIT 20
```

Detailed findings - *table*

```
select * from monitoring.v_SEC_SQL_ACC_011_RC02 order by entry_date desc limit 100
```

Root cause details - ${root*cause*id} - *table*

```
SELECT j.value AS result FROM jsonb_array_elements(
COALESCE(monitoring.get_root_cause_resultset(NULLIF('${root_cause_id}',''), $__timeFrom())::jsonb, '[]'::jsonb) ) AS j LIMIT
500
```

Super users (sysadmin / sysDBA)

SYSADMIN / SYSDBA - *barchart*

```
select count(*) , server from monitoring.v_mssql_superuser where login_name != 'infosecuser' group by server
```

Active users - *bargauge*

```
select sum("Number of connections") from ( select count(*) "Number of connections" from monitoring.tcp_connections where
connect_time between Now() - interval '1 minute' and Now() union all select count(*) from monitoring.v_oracle_transactions
where entry_date > Now() - interval '1 minute' )
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_sysadmin
```

Recent alerts - *stat*

```
select (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and
query ILIKE ANY (ARRAY['%/*%*/%', '%--%'] )) as "comment_based_injections", (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%OR
1=1%', '%AND 1=1%' , '%OR ''a''=''a%''' ])) as "Boolean-based injections" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
1=1 --''%', '%"" OR 1=1 --%'])) as "tautology_with_comments" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%UNION SELECT', '%UNION ALL SELECT%'])) as
"union_based_injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now() - interval
'1 minute' and query ILIKE ANY (ARRAY['%; DROP TABLE users', '%; EXEC xp_cmdshell%'])) as "stacked_queries" , (select count(*)
from monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%WAITFOR DELAY%', '%pg_sleep%','%SLEEP%'])) as "time_based_blind _injection" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%CONVERT%', '%CAST%'])) as "error_based_injection" , (select count(*) from monitoring.active_transactions where
LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%information_schema.tables%',
'%information_schema.columns%','%sys.tables%','%pg_catalog%'])) as "information_schema_probing" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%'' OR
''=''%', '%'' OR ''x''=''x''%', '%admin''--%'])) as "Authentication bypass patterns" , (select count(*) from
monitoring.active_transactions where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY
(ARRAY['%0x414243%', '%CHAR(65,66,67)%'])) as "encoding injection" , (select count(*) from monitoring.active_transactions
where LAST_request_end_time > Now() - interval '1 minute' and query ILIKE ANY (ARRAY['%xp_cmdshell%', '%sp_executesql%'])) as
"shell_system_functions injection" , (select count(*) from monitoring.active_transactions where LAST_request_end_time > Now()
- interval '1 minute' and query ILIKE ANY (ARRAY['%ORDER BY 1%', '%ORDER BY%'])) as "ORDER BY_GROUP BY injection" from
monitoring.active_transactions group by server
```

Credentials stored in database tables - *table*

```
select * from monitoring.v_mssql_superuser where login_name != 'infosecuser'
```

Take Action

Critical issues - SQL injection - *gauge*

```
SELECT SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time = TRUE THEN
1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS boolean_sqli , SUM(CASE WHEN
long_literal = TRUE THEN 1 ELSE 0 END) AS long_literal , SUM(CASE WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS outfile_copy
, SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS union_select , SUM(CASE WHEN schema_access = TRUE THEN 1 ELSE 0 END)
AS schema_access , SUM(CASE WHEN stacked_query = TRUE THEN 1 ELSE 0 END) AS stacked_query , SUM(CASE WHEN commented_payload =
TRUE THEN 1 ELSE 0 END) AS commented_payload , SUM(CASE WHEN sensitive_columns = TRUE THEN 1 ELSE 0 END) AS sensitive_columns
FROM monitoring.v_sql_feature_predictions;
```

Active threats - *gauge*

```
select count(*) "suspicious queries" from monitoring.v_suspiscous
```

Active users - *bargauge*

```
select count(*) from monitoring.tcp_connections where connect_time between Now - interval '1 month' and Now()
```

Take action - *table*

```
select row_id , action_name, action_description, server, case when is_active = true then 'Enabled' else 'disabled' end Enabled
from config.action_types where server != 'unknown' order by server , action_name
```

Recent alerts - *stat*

```
select SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time = TRUE THEN
1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS boolean_sqli , SUM(CASE WHEN
long_literal = TRUE THEN 1 ELSE 0 END) AS long_literal , SUM(CASE WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS outfile_copy
, SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS union_select , SUM(CASE WHEN schema_access = TRUE THEN 1 ELSE 0 END)
AS schema_access , SUM(CASE WHEN stacked_query = TRUE THEN 1 ELSE 0 END) AS stacked_query , SUM(CASE WHEN commented_payload =
TRUE THEN 1 ELSE 0 END) AS commented_payload , SUM(CASE WHEN sensitive_columns = TRUE THEN 1 ELSE 0 END) AS sensitive_columns
FROM monitoring.v_sql_feature_predictions
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

Connectivity reports - *barchart*

```
select count(*) , server ,client_net_address from monitoring.tcp_connections where connect_Time > Now() - interval '1 hour'
group by server ,client_net_address
```

Transaction report

(untitled panel) - *table*

```
select distinct server from monitoring.v_blocking_transactions
```

(untitled panel) - *table*

```
select TO_CHAR(to_timestamp(start_time::bigint / 1000 ), 'YYYY-MM-DD') AS "start time" from monitoring.v_blocking_transactions
order by TO_CHAR(to_timestamp(start_time::bigint / 1000 ), 'YYYY-MM-DD') limit 1
```

(untitled panel) - *table*

```
select TO_CHAR(to_timestamp(start_time::bigint / 1000 ), 'YYYY-MM-DD') AS "start time" from monitoring.v_blocking_transactions
order by TO_CHAR(to_timestamp(start_time::bigint / 1000 ), 'YYYY-MM-DD') limit 1
```

Transactions - *table*

```
select * from monitoring.v_combined_transactions WHERE $__timeFilter(last_request_end_time) order by last_request_end_time
desc
```

```
select SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) + SUM(CASE WHEN sleep_time = TRUE THEN 1 ELSE 0 END) +
SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN long_literal = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN
outfile_copy = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END)+ SUM(CASE WHEN schema_access =
TRUE THEN 1 ELSE 0 END) + SUM(CASE WHEN stacked_query = TRUE THEN 1 ELSE 0 END) + SUM(CASE WHEN commented_payload = TRUE THEN
1 ELSE 0 END) + SUM(CASE WHEN sensitive_columns = TRUE THEN 1 ELSE 0 END) "Active threats" FROM
monitoring.v_sql_feature_predictions
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

Connectivity reports - *barchart*

```
select count(*) , server ,client_net_address from monitoring.tcp_connections where connect_Time > Now() - interval '1 hour'
group by server ,client_net_address
```

Vulnerability

SQL Injections - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-INJ%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Suspicious Queries - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-ACC-010-RC04%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Config Vulnerabilities - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL-CFG%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Active Users - *stat*

```
SELECT COUNT(*) FROM monitoring.tcp_connections WHERE $__timeFilter(connect_time)
```

Total Findings - *stat*

```
SELECT COUNT(*) FROM monitoring.general_metric_metadata_results WHERE metric_name LIKE 'SEC-SQL%%' AND
(metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(entry_date)
```

Vulnerability by Issue Type - *barchart*

```
SELECT COALESCE(i.name, gm.metric_name) AS issue, COUNT(*) AS findings FROM monitoring.general_metric_metadata_results gm LEFT
JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id
WHERE gm.metric_name LIKE 'SEC-SQL%%' AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND
```

```
$__timeFilter(gm.entry_date) GROUP BY issue ORDER BY findings DESC LIMIT 15
```

By Security Area - *piechart*

```
SELECT COALESCE(a.name, 'Unknown') AS area, COUNT(*) AS count FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id LEFT JOIN rootcause.areas a ON a.code = i.area_code WHERE gm.metric_name LIKE 'SEC-SQL%%' AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY area ORDER BY count DESC
```

Security Vulnerability Findings - *table*

```
SELECT gm.server, COALESCE(a.name, '') AS area, COALESCE(i.name, '') AS issue, gm.metric_name AS root_cause_id, COALESCE(rc.name, gm.metric_name) AS root_cause, (gm.metric_metadata_vs_expected::jsonb->>'row_count')::int AS rows, COALESCE(gm.metric_metadata_vs_expected::jsonb->>'severity', 'medium') AS severity, (gm.metric_metadata_vs_expected::jsonb->'expected'->>'description') AS description, gm.entry_date AT TIME ZONE 'Asia/Jerusalem' AS detected_at FROM monitoring.general_metric_metadata_results gm LEFT JOIN rootcause.root_causes rc ON rc.root_cause_id = gm.metric_name LEFT JOIN rootcause.issues i ON i.issue_id = rc.issue_id LEFT JOIN rootcause.areas a ON a.code = i.area_code WHERE gm.metric_name LIKE 'SEC-SQL%%' AND gm.metric_metadata_vs_expected IS NOT NULL AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) ORDER BY gm.entry_date DESC LIMIT 100
```

SQL Injection by Server - *barchart*

```
SELECT gm.server, COUNT(*) AS injections FROM monitoring.general_metric_metadata_results gm WHERE gm.metric_name LIKE 'SEC-SQL-INJ%%' AND (gm.metric_metadata_vs_expected::jsonb->>'matched')::text = 'true' AND $__timeFilter(gm.entry_date) GROUP BY gm.server ORDER BY injections DESC LIMIT 10
```

Connectivity by Server - *barchart*

```
SELECT server || ' <- ' || client_net_address AS client, COUNT(*) AS connections FROM monitoring.tcp_connections WHERE $__timeFilter(connect_time) GROUP BY server, client_net_address ORDER BY connections DESC LIMIT 10
```

Webhook Alerts Configuration

Webhook Alerts - click a true/false cell to toggle - *table*

```
SELECT row_id, metric_type, risk_level, send_mail_alert, send_siem_alert, send_diagnosis_evidence, blocker, auto_mask, is_active, recurrency_hours, (NOT send_mail_alert) AS mail_next, (NOT send_siem_alert) AS siem_next, (NOT send_diagnosis_evidence) AS diag_next, (NOT blocker) AS blocker_next, (NOT auto_mask) AS auto_mask_next, (NOT is_active) AS active_next FROM config.webook_alerts ORDER BY metric_type, risk_level
```

Blocker global dry-run (true = log only, false = ARMED to kill) - *table*

```
SELECT value AS blocker_dry_run, CASE WHEN value='true' THEN 'false' ELSE 'true' END AS next_val FROM config.global_params WHERE key='blocker_dry_run' ORDER BY row_id DESC LIMIT 1
```

Masking global dry-run (true = log only, false = ARMED to apply masks) - *table*

```
SELECT value AS masking_dry_run, CASE WHEN value='true' THEN 'false' ELSE 'true' END AS next_val FROM config.global_params WHERE key='masking_dry_run' ORDER BY row_id DESC LIMIT 1
```

alert_report

Open Alerts - *nodeGraph*

```
select * from flowchart.v_sources_alerts
```

Open Alerts - *table*

```
SELECT server , root_cause , query , entry_date FROM monitoring.alerts_open WHERE state = 'open'
```

alert*report*Error_Based

Open Alerts - *nodeGraph*

```
select * from flowchart.v_sources_alerts
```

Open Alerts - *table*

```
SELECT server , root_cause , query , entry_date FROM monitoring.alerts_open WHERE state = 'open' and root_cause = 'Error-Based Injection (MSSQL-style)'
```

alert*report*Privilege*Escalation*Chain

Open Alerts - *nodeGraph*

```
select * from flowchart.v_sources_alerts
```

Open Alerts - *table*

```
SELECT server , root_cause , query , entry_date FROM monitoring.alerts_open WHERE state = 'open'
```

alert*report*Stacked

Open Alerts - *nodeGraph*

```
select * from flowchart.v_sources_alerts
```

Open Alerts - *table*

```
SELECT server , root_cause , query , entry_date FROM monitoring.alerts_open WHERE state = 'open' and root_cause = 'Stacked Queries (Very MSSQL-Specific)'
```

alert*report*sensitive_schema

Open Alerts - *nodeGraph*

```
select * from flowchart.v_sources_alerts
```

Sensitive columns and schema (PII) - *table*

```
select distinct server , table_catalog , table_name , column_name entdtry_date from monitoring.schema
```

```
select susp.server, duration_secs, last_request_end_time, command, program_name, query from monitoring.v_combined_transactions susp join monitoring.sensitive_schema sc on susp.query like '%'||sc.column_name ||'%' where last_request_end_time > $__timeFrom(last_request_end_time)
```

```
select server , table_name , column_name ,columns , query from ( select row_number() over ( partition by query order by p.entry_date desc ) seq , p.server , sc.table_name , sc.column_name ,columns , p.condition , joins , func , p.query from monitoring.metric_query_parsing p join monitoring.sensitive_schema sc on p.query like '%'||sc.column_name ||'%' ) where seq = 1
```

Sensitive data (PII) - *table*

```
select* from monitoring.v_sensitive_data_inuse where $__timeFilter(last_request_end_time)
```

risk level alerts

Risk level alerts - *table*

```
select distinct rc.issue_name , rc.root_cause_name , rc.Risk_level , rl.is_active from rootcause.v_rootcauses rc join
rootcause.risk_level rl on rl.risk_level = rc.risk_level and domain_code = 'SEC'
```

Risk Levels - *table*

```
SELECT row_id, trim(risk_level) AS risk_level, is_active FROM rootcause.risk_level ORDER BY row_id
```

sysadmin accounts with weak password enforcement

sysadmin accounts with weak password enforcement - *barchart*

```
select server , count(*) lock_date from monitoring.v_sysadmin_accounts_with_weakpassword_enforcement group by server
```

(untitled panel) - *table*

```
select distinct server from monitoring.v_sysadmin_accounts_with_weakpassword_enforcement
```

sysadmin accounts with weak password enforcement - *table*

```
select * from monitoring.v_sysadmin_accounts_with_weakpassword_enforcement
```

Sensitivity reports - *barchart*

```
SELECT server , count(*) "Number of sensitivity issues" FROM ( SELECT query_id , entry_date , server ,
(sql_feature_predictions.features ->> 'query'::text) AS query, ((sql_feature_predictions.features ->>
'sensitive_columns'::text))::boolean AS sensitive_columns FROM monitoring.sql_feature_predictions) a left outer join
monitoring.sql_suspicious susp on susp.query_id = a.query_id WHERE (sensitive_columns IS TRUE) group by server
```

SQL Injection analysis - *barchart*

```
SELECT SERVER , SUM(CASE WHEN dangerous_function = TRUE THEN 1 ELSE 0 END) AS dangerous_function, SUM(CASE WHEN sleep_time =
TRUE THEN 1 ELSE 0 END) AS sleep_time , SUM(CASE WHEN boolean_sqli = TRUE THEN 1 ELSE 0 END) AS "boolean Injection" , SUM(CASE
WHEN outfile_copy = TRUE THEN 1 ELSE 0 END) AS "copy injections" , SUM(CASE WHEN union_select = TRUE THEN 1 ELSE 0 END) AS
"union_select injections" , SUM(CASE WHEN commented_payload = TRUE THEN 1 ELSE 0 END) AS commented_payload FROM
monitoring.v_sql_feature_predictions GROUP BY SERVER
```

Data activity monitoring - *barchart*

```
select count(*) , server from monitoring.active_transactions where last_request_end_time > Now() - interval '1 hour' group by
server
```

webhook alerts

web hook alerts - *table*

```
SELECT row_id, metric_type, send_mail_alert, send_siem_alert, send_diagnosis_evidence FROM config.webook_alerts ORDER BY
metric_type
```